CS 226 Section 3

Database Management

Instructor:

Ratana Soth

Topic:

Parking Management System

Parkin'

Parking Management System

Final Documentation

Course	CS226 — Database
Institution	Paragon International University
Team	Kibriyo Shomirzoeva · Heng Sok · Dyvannda San
Stack	Laravel 11 · MySQL 8 · Bootstrap 5.3.3

Part 1 — Project Description

1.1 Project Overview

Parkin' is a full-stack web-based parking management system built for a real parking lot operation. It replaces manual paper-based tracking with a complete digital workflow covering vehicle entry, zone-based slot assignment, fee calculation, payment collection, parking history, and financial reporting.

The system supports two roles: Staff who handle day-to-day operations, and Admin who additionally manage staff accounts, view financial reports, cancel tickets, and configure fee rates.

1.2 Team Members

Name	Student ID	Department	Contact
Kibriyo Shomirzoeva	230201104	CS Department	kshomirzoeva@paragoniu.edu.kh
Heng Sok	240201012	CS Department	hsok1@paragoniu.edu.kh
Dyvannda San	230105002	IE Department	dsan@paragoniu.edu.kh

1.3 Technology Stack

Layer	Technology	Purpose
Backend	Laravel 11 (PHP)	MVC framework, Eloquent ORM, routing, middleware
Database	MySQL 8 (port 3307)	Relational data storage, FK constraints, indexes
Frontend	Bootstrap 5.3.3 + Bootstrap Icons	Responsive UI components
Font	Inter (Google Fonts)	Clean typography
Authentication	Custom — no Breeze or Sanctum	Built from scratch for custom staff table

Layer	Technology	Purpose
File Storage	Laravel Storage public disk	Profile image uploads
Dev Environment	Laravel Herd	Local server and MySQL

1.4 Role Access

Feature	Staff	Admin
Dashboard	Yes	Yes
Vehicle Check-In	Yes	Yes
Vehicle Check-Out	Yes	Yes
Slot Map (Aerial + List)	Yes	Yes
Vehicle Management (CRUD)	Yes	Yes
Parking History + CSV Export	Yes	Yes
Fee Rates	View only	View + Edit
Reports + Revenue Analytics	No	Yes
Staff Management	No	Yes
Cancel Active Ticket	No	Yes
Toggle Slot Maintenance	Yes	Yes
Settings (Profile + Password)	Yes	Yes

1.5 Features and Functions

1. Authentication

Staff log in with a username and password. The system uses a custom staff table with a password_hash column. Deactivated accounts are blocked at login even with the correct password. Sessions are regenerated after successful login.

2. Dashboard

Real-time overview of slot occupancy, today's revenue, total transactions, and an hourly traffic bar chart. Period filter tabs: Today, Week, Month, Year. Traffic chart uses a single GROUP BY HOUR() query instead of 24 separate queries.

3. Vehicle Check-In

Staff select vehicle type and enter a plate number. Structured plates are validated: prefix 2 for cars, prefix 1 for motorcycles and tricycles, 2 letters, dash, 4 digits (e.g. 2AB-1234). Bikes auto-generate BIKE-001, BIKE-002 etc. Staff then select a slot from a real-time colour-coded grid. Only the correct zone is clickable. All operations run inside DB::transaction(). A ticket with unique ID and barcode is issued. Staff can correct a wrong plate from the ticket page without cancelling.

4. Vehicle Check-Out

Staff search by plate number or ticket ID. A live timer shows the estimated fee updating in real time. Fee is recalculated server-side at payment to prevent browser manipulation. Staff select a payment method (Cash, Card, QR Scan) and confirm. Inside a transaction with lockForUpdate(), the ticket is marked completed, the slot is freed, and a payment record is inserted. Admin can cancel an active ticket from this page without payment.

5. Slot Map

Two views: Aerial (colour-coded slot grid per zone) and List (searchable by plate or slot number). Both views compute slot status from active tickets in real time — not from the slot.status column which can go stale. Admin can toggle maintenance mode from the slot detail page.

6. Vehicle Management

Full CRUD for vehicles. Searching by plate shows a red Currently Parked panel if the vehicle is active, with direct buttons to View Ticket, Edit Plate, and Check Out. Removing a vehicle soft-deletes it if it has parking history (status = deleted), preserving the audit trail.

7. Parking History

Complete log of all sessions. Filterable by period, status (Active, Completed, Cancelled), and vehicle type. Searchable by plate number. CSV export available. Clicking a plate shows the full history for that vehicle.

8. Fee Rates

Admin can edit flat-rate fees per vehicle type. Two rates per type: short stay (under 5 hours) and long stay (5 hours or more). Fees shown in USD with KHR equivalent at 1 USD = 4,000 KHR.

9. Reports (Admin Only)

Revenue analytics with period and type filters. Revenue trend line chart, vehicle type donut chart, and top 5 most frequent vehicles using correlated subqueries. CSV export available.

10. Staff Management (Admin Only)

Admin can add, edit, deactivate, and permanently delete staff accounts. New accounts default password is password. Admin can reset any staff password. Admin cannot delete their own account.

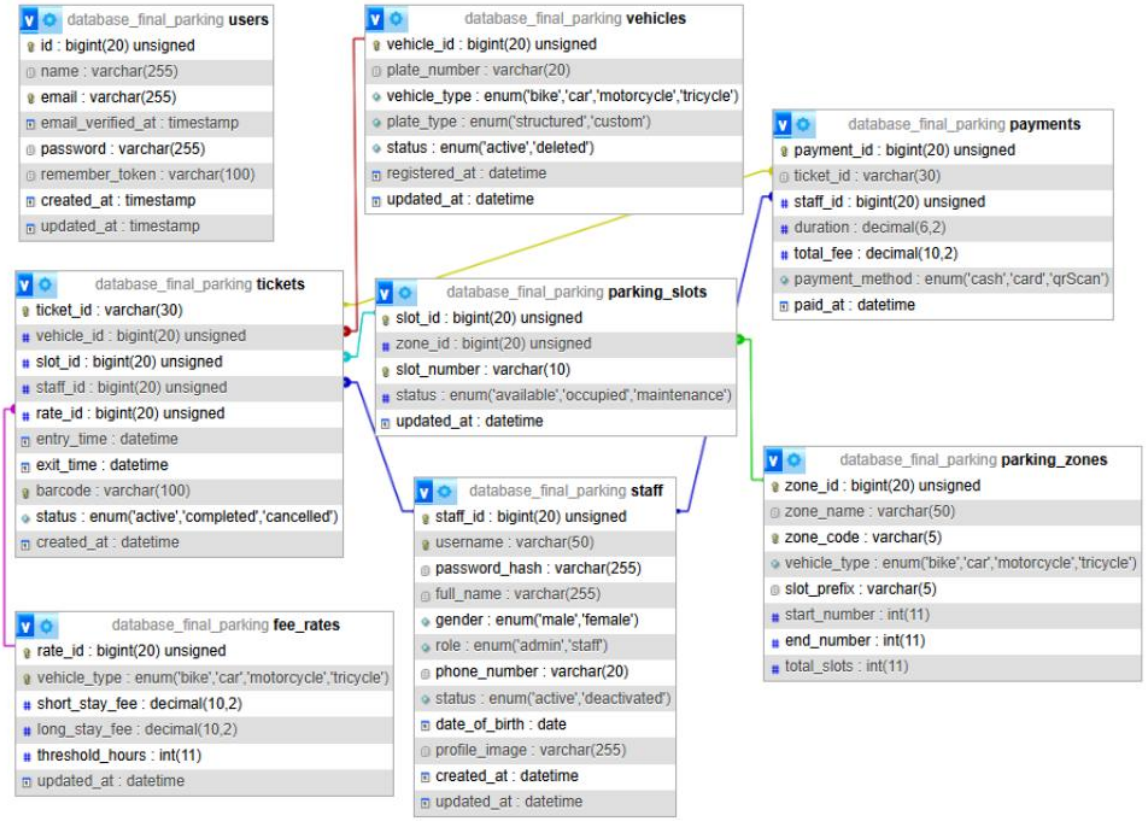
11. Settings

All staff can update profile info and change their password. Password change requires the current password.

12. Correct Wrong Plate

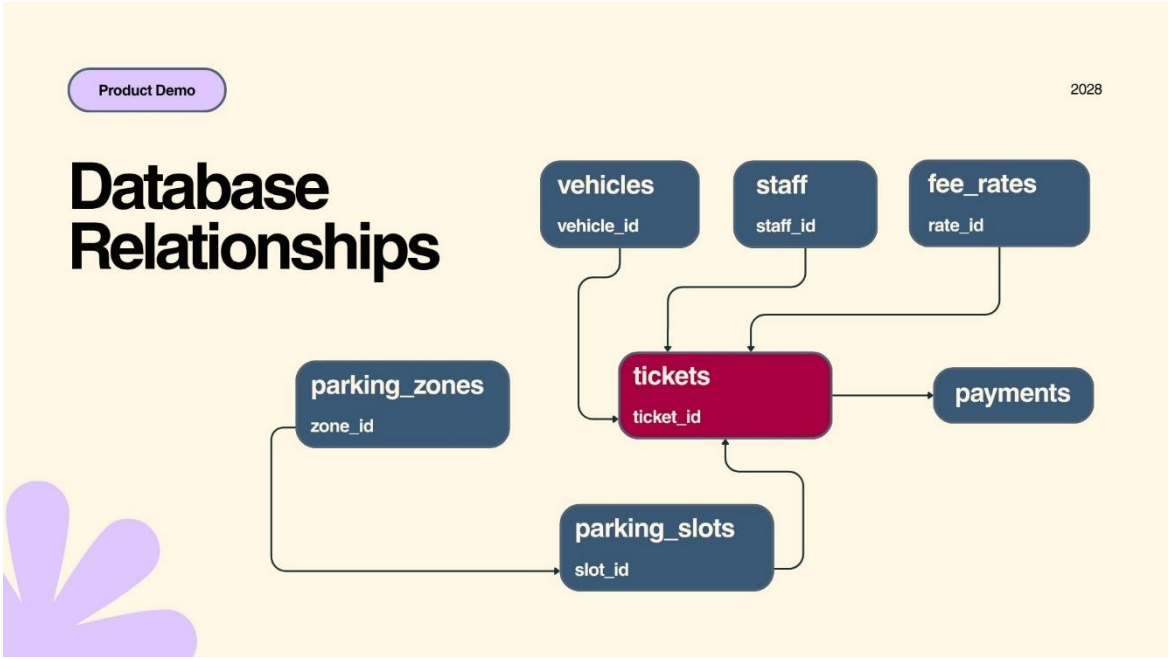
If staff check in the wrong plate, they correct it from the ticket page or slot detail page. If the correct plate exists as another vehicle, the ticket is reassigned to it. If not, the plate is updated. Slot, entry time, and ticket stay unchanged.

Part 2 — Database Schema



7 application tables with explicitly named foreign key constraints and indexes. The tickets table is the central record connecting vehicle, slot, staff, and fee rate.

2.1 Entity Relationship Overview



2.2 Table: staff

Column	Type	Constraint	Description
staff_id	BIGINT UNSIGNED	PK AUTO_INCREMENT	Unique staff identifier
username	VARCHAR(50)	UNIQUE NOT NULL	Login username
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hash — custom column name, requires model override
full_name	VARCHAR(255)	NOT NULL	Display name
gender	ENUM(male,female)	NOT NULL	Gender
role	ENUM(admin,staff)	NOT NULL	Access level
phone_number	VARCHAR(20)	NOT NULL	Contact number
status	ENUM(active,deactivated)	DEFAULT active	Login access control
date_of_birth	DATE	NOT NULL	Date of birth

Column	Type	Constraint	Description
profile_image	VARCHAR(255)	NULLABLE	Storage path for uploaded photo
created_at	DATETIME	NOT NULL	Record creation timestamp
updated_at	DATETIME	NOT NULL	Last update timestamp

2.3 Table: vehicles

Column	Type	Constraint	Description
vehicle_id	BIGINT UNSIGNED	PK AUTO_INCREMENT	Unique vehicle identifier
plate_number	VARCHAR(20)	NULLABLE	Null for bikes — auto-generated as BIKE-XXX
vehicle_type	ENUM(bike,car,motorcycle,tricycle)	NOT NULL	Vehicle category
plate_type	ENUM(structured,custom)	DEFAULT structured	Structured enforces format; custom is free-form
status	ENUM(active,deleted)	DEFAULT active	Soft delete — history preserved when deleted
registered_at	DATETIME	NOT NULL	Registration timestamp
updated_at	DATETIME	NOT NULL	Last update timestamp

2.4 Table: parking_zones

Column	Type	Constraint	Description
zone_id	BIGINT UNSIGNED	PK AUTO_INCREMENT	Unique zone identifier
zone_name	VARCHAR(50)	NOT NULL	Display name (e.g. Zone A — Car)

Column	Type	Constraint	Description
zone_code	VARCHAR(5)	UNIQUE NOT NULL	Prefix for slot number generation (A, M, T, K)
vehicle_type	ENUM(bike,car,motorcycle,tricycle)	NOT NULL	Accepted vehicle type — lives HERE, not on parking_slots
slot_prefix	VARCHAR(5)	NOT NULL	Slot number prefix
start_number	INT	NOT NULL	Starting slot number for seeder
end_number	INT	NOT NULL	Ending slot number for seeder
total_slots	INT	NOT NULL	Total slots in this zone

4 zones in the system:

Zone	zone_id	vehicle_type	Slot Range	Total
CAR	1	car	A01 — A30	30
MOTORCYCLE	2	motorcycle	M01 — M15	15
TRICYCLE	3	tricycle	T01 — T05	5
BIKE	4	bike	K01 — K08	8

2.5 Table: parking_slots

Column	Type	Constraint	Description
slot_id	BIGINT UNSIGNED	PK AUTO_INCREMENT	Unique slot identifier
zone_id	BIGINT UNSIGNED	FK → parking_zones CASCADE	Which zone this slot belongs to
slot_number	VARCHAR(10)	UNIQUE NOT NULL	Human-readable label (A01, M03, K08)

Column	Type	Constraint	Description
status	ENUM(available,occupied,maintenance)	DEFAULT available	Physical status — display always uses real_status from tickets
updated_at	DATETIME	NOT NULL	Last status change timestamp

2.6 Table: fee_rates

Column	Type	Constraint	Description
rate_id	BIGINT UNSIGNED	PK AUTO_INCREMENT	Unique rate identifier
vehicle_type	ENUM(bike,car,motorcycle,tricycle)	UNIQUE NOT NULL	One rate row per vehicle type
short_stay_fee	DECIMAL(10,2)	NOT NULL	USD fee for stays under threshold_hours
long_stay_fee	DECIMAL(10,2)	NOT NULL	USD fee for stays at or over threshold_hours
threshold_hours	INT	DEFAULT 5	Hour boundary between short and long stay
updated_at	DATETIME	NOT NULL	Last rate update

Current rates (1 USD = 4,000 KHR):

Vehicle Type	Short Stay Under 5h	Long Stay 5h+	KHR Short	KHR Long
Car	$1.25	$5.00	5,000 KHR	20,000 KHR
Motorcycle	$0.25	$1.00	1,000 KHR	4,000 KHR
Bike	$0.25	$1.00	1,000 KHR	4,000 KHR
Tricycle	$1.00	$2.00	4,000 KHR	8,000 KHR

2.7 Table: tickets

Column	Type	Constraint	Description
ticket_id	VARCHAR(30)	PK (string)	Generated as T + 6 uppercase chars from uniqid() e.g. T3DAFF2
vehicle_id	BIGINT UNSIGNED	FK → vehicles RESTRICT	Which vehicle is parked
slot_id	BIGINT UNSIGNED	FK → parking_slots RESTRICT	Which slot is occupied
staff_id	BIGINT UNSIGNED	FK → staff RESTRICT	Which staff processed the check-in
rate_id	BIGINT UNSIGNED	FK → fee_rates RESTRICT	Which fee rate applies
entry_time	DATETIME	NOT NULL	When the vehicle entered
exit_time	DATETIME	NULLABLE	When the vehicle exited — null while active
barcode	VARCHAR(100)	UNIQUE	Same value as ticket_id — used to render CODE128 barcode
status	ENUM(active,completed,cancelled)	DEFAULT active	Ticket lifecycle status
created_at	DATETIME	NOT NULL	Record creation timestamp

Indexes: status, vehicle_id, slot_id, staff_id, entry_time

2.8 Table: payments

Column	Type	Constraint	Description
payment_id	BIGINT UNSIGNED	PK AUTO_INCREMENT	Unique payment identifier
ticket_id	VARCHAR(30)	FK → tickets CASCADE	Which ticket was paid — cascades on ticket delete
staff_id	BIGINT UNSIGNED	FK → staff RESTRICT	Which staff processed the payment

Column	Type	Constraint	Description
duration	DECIMAL(6,2)	NOT NULL	Parking duration in decimal hours
total_fee	DECIMAL(10,2)	NOT NULL	Amount charged in USD
payment_method	ENUM(cash,card,qrScan)	NOT NULL	How the customer paid
paid_at	DATETIME	NOT NULL	Exact payment timestamp

Indexes: paid_at, staff_id

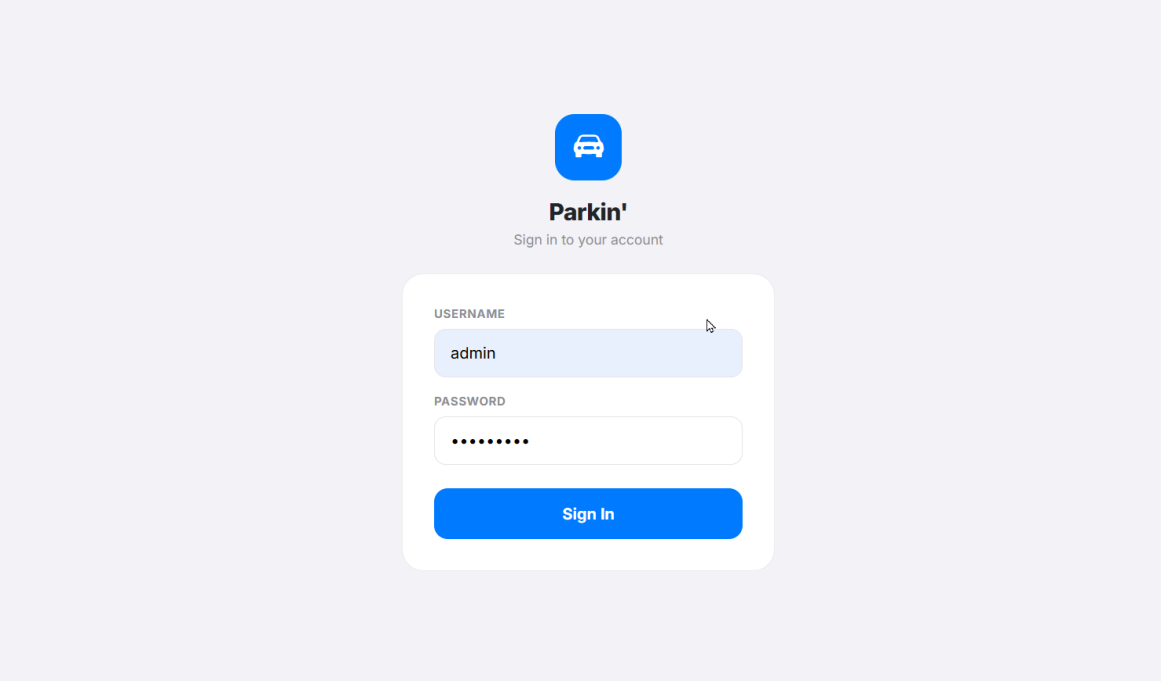
2.9 Named Foreign Keys and Indexes

Name	Table	Type	Column	References / Action
fk_slots_zone	parking_slots	Foreign Key	zone_id	parking_zones.zone_id — CASCADE
idx_slots_status	parking_slots	Index	status	—
fk_tickets_vehicle	tickets	Foreign Key	vehicle_id	vehicles.vehicle_id — RESTRICT
fk_tickets_slot	tickets	Foreign Key	slot_id	parking_slots.slot_id — RESTRICT
fk_tickets_staff	tickets	Foreign Key	staff_id	staff.staff_id — RESTRICT
fk_tickets_rate	tickets	Foreign Key	rate_id	fee_rates.rate_id — RESTRICT
idx_tickets_status	tickets	Index	status	—
idx_tickets_vehicle	tickets	Index	vehicle_id	—
idx_tickets_slot	tickets	Index	slot_id	—

Name	Table	Type	Column	References / Action
idx_tickets_staff	tickets	Index	staff_id	—
idx_tickets_entry	tickets	Index	entry_time	—
fk_payments_ticket	payments	Foreign Key	ticket_id	tickets.ticket_id — CASCADE
fk_payments_staff	payments	Foreign Key	staff_id	staff.staff_id — RESTRICT
idx_payments_paid_at	payments	Index	paid_at	—
idx_payments_staff	payments	Index	staff_id	—

Part 3 — Prototype with SQL

3.1 Login

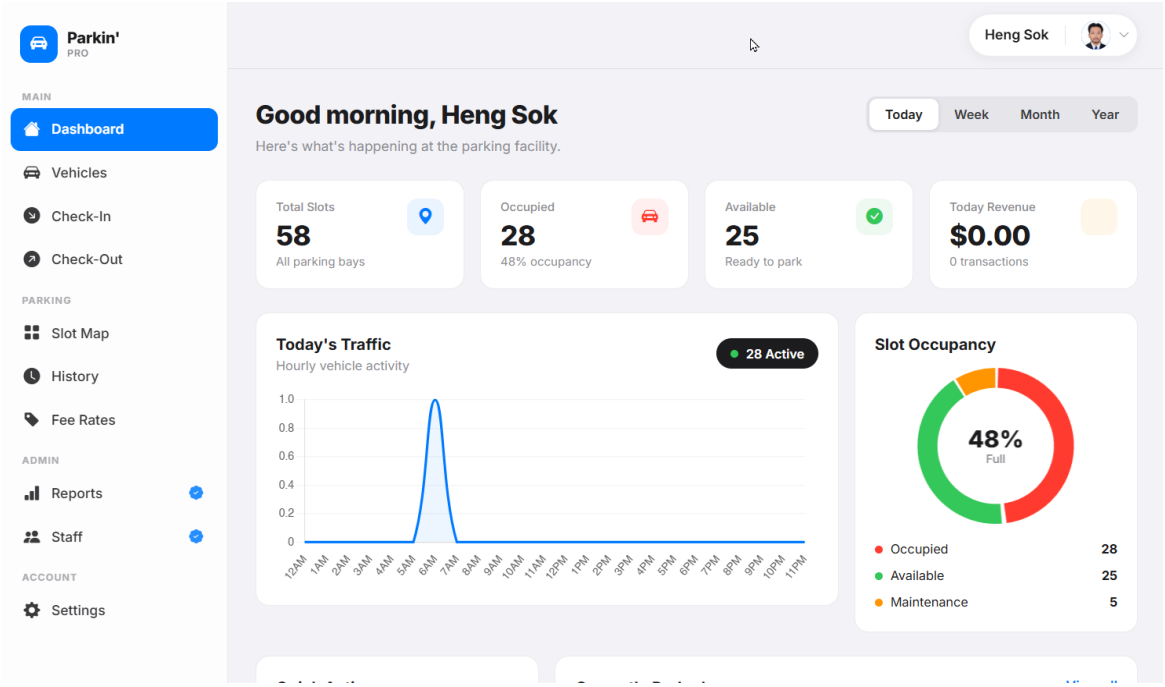


Related Tables	staff
Purpose	Authenticate a staff member using username and password. The system reads from the staff table using a custom password_hash column. Deactivated accounts are rejected even with correct credentials.

SQL Statement	`SELECT *`
	`FROM staff`
	`WHERE username = 'admin'`
	`LIMIT 1;`
	
	`UPDATE staff`
	`SET updated_at = NOW()`
	`WHERE staff_id = 1;`

Explanation	The controller queries the staff table by username. It verifies the password against password_hash using Laravel's Hash::check(). If status is deactivated, login is blocked. On success the session is regenerated to prevent fixation attacks.
Expected Result	Valid active credentials redirect to /dashboard. Wrong password shows Invalid username or password. Deactivated account shows Your account has been deactivated.

3.2 Dashboard



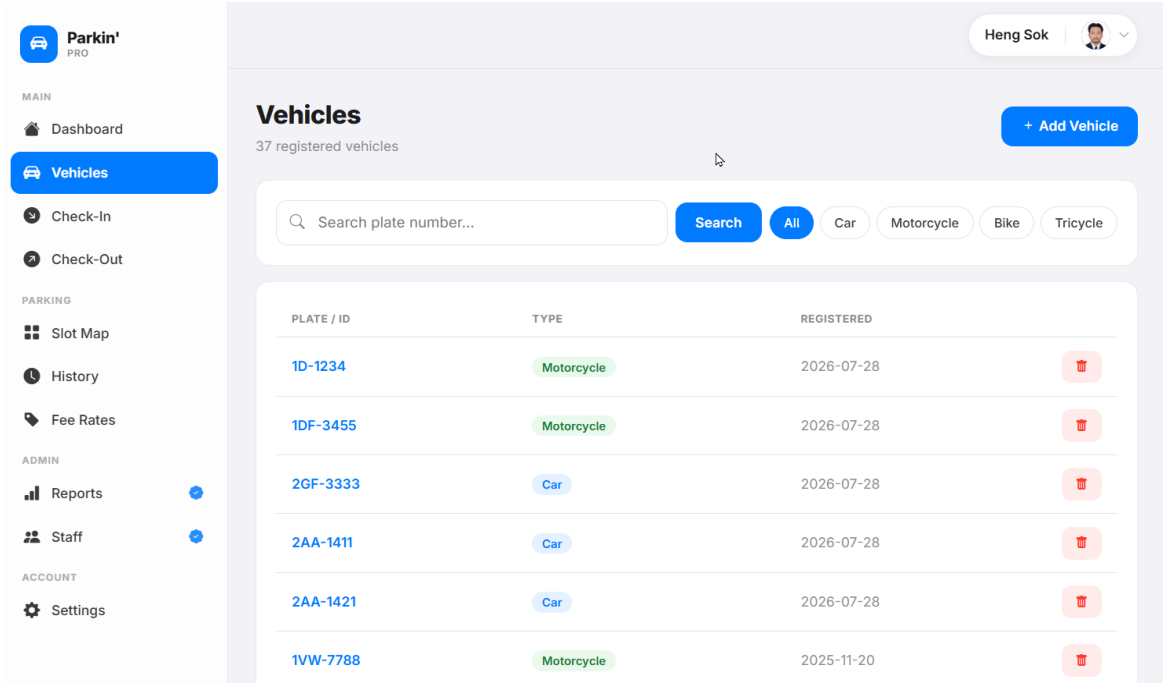
Related Tables	tickets, payments, parking_slots
Purpose	Show real-time parking stats and an hourly traffic chart. Period filter tabs change the revenue aggregation window.

SQL Statement	`SELECT *`
	`FROM parking_slots;`
	
	`SELECT *`
	`FROM tickets`
	`WHERE status = 'active';`
	
	`SELECT SUM(total_fee),`
	`COUNT(*)`
	`FROM payments`
	`WHERE paid_at >= '2026-07-30 00:00:00';`
	
	`SELECT HOUR(entry_time) AS hour,`
	`COUNT(*) AS count`
	`FROM tickets`
	`WHERE DATE(entry_time) = '2026-07-30'`
	`GROUP BY hour;`

Explanation	Slot occupancy is computed in PHP from active tickets — not slot.status — to prevent stale data. Revenue is aggregated by paid_at date filter. The traffic chart uses one GROUP BY HOUR() query and PHP fills missing hours with 0.

Expected Result	Dashboard shows 58 Total Slots, Occupied count, Available count, Maintenance count, today's revenue, total transactions, hourly bar chart, and the 6 most recent active vehicles.

3.3 Vehicle List

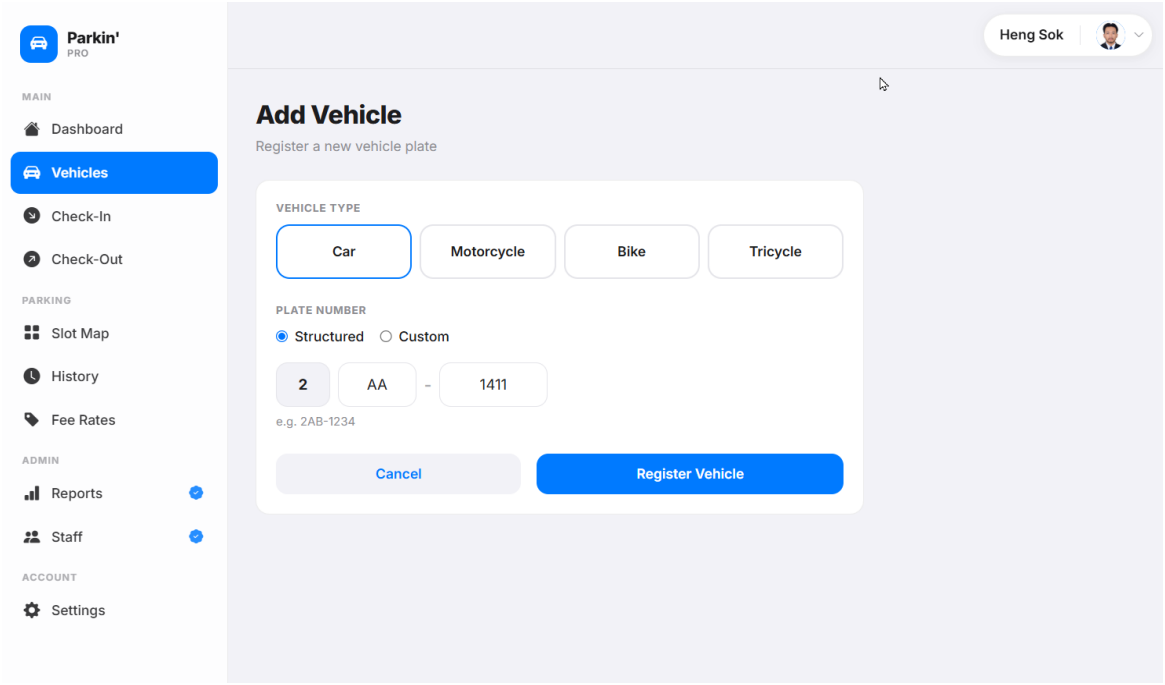


Related Tables	vehicles
Purpose	Show all active vehicles with search by plate number and filter by type. Paginated 20 per page.

SQL Statement	`SELECT *`
	`FROM vehicles`
	`WHERE status = 'active'`
	`AND plate_number LIKE '%2AB%'`
	`AND vehicle_type = 'car'`
	`ORDER BY registered_at DESC`
	`LIMIT 20 OFFSET 0;`

Explanation	Only active vehicles are returned. The search applies a LIKE filter on plate_number. The type filter adds a vehicle_type condition. Results are paginated and ordered by registration date descending.
Expected Result	Table shows matching active vehicles with plate, type badge, registered date, and a remove button. Pagination links appear if results exceed 20.

3.4 Add Vehicle

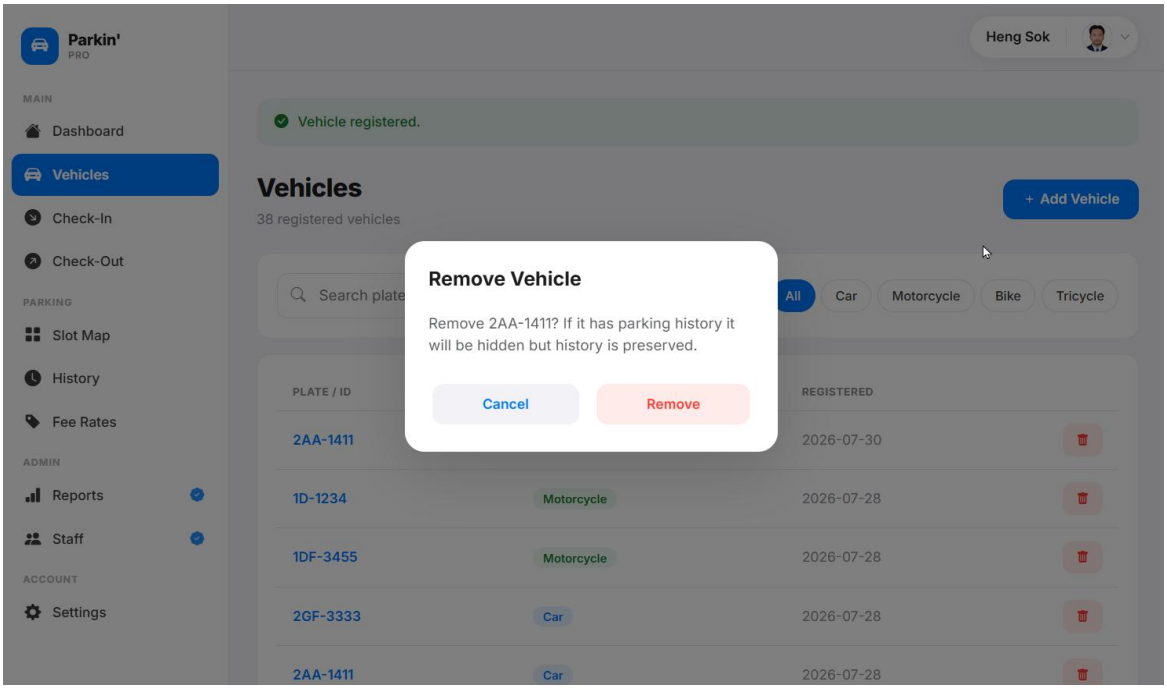


Related Tables	vehicles
Purpose	Register a new vehicle with plate number and type. Structured plates are format-validated client-side and server-side. Bike plates are auto-generated.

SQL Statement	`SELECT plate_number`
	`FROM vehicles`
	`WHERE vehicle_type = 'bike'`
	`AND plate_number LIKE 'BIKE-%'`
	`ORDER BY vehicle_id DESC`
	`LIMIT 1;`
	
	`INSERT INTO vehicles`
	`(plate_number, vehicle_type, plate_type,`
	`status, registered_at, updated_at)`
	`VALUES ('2AA-1411', 'car', 'structured',`
	`'active', NOW(), NOW());`

Explanation	For bikes, the last BIKE-XXX number is fetched and incremented. For structured plates, server-side regex /^[12][A-Z]{2}-\d{4}$/ validates the format before INSERT. Frontend JS also enforces this on each input field.
Expected Result	New vehicle record inserted. Redirect to vehicle list with Vehicle registered. toast. The vehicle is now available for check-in.

3.5 Remove Vehicle

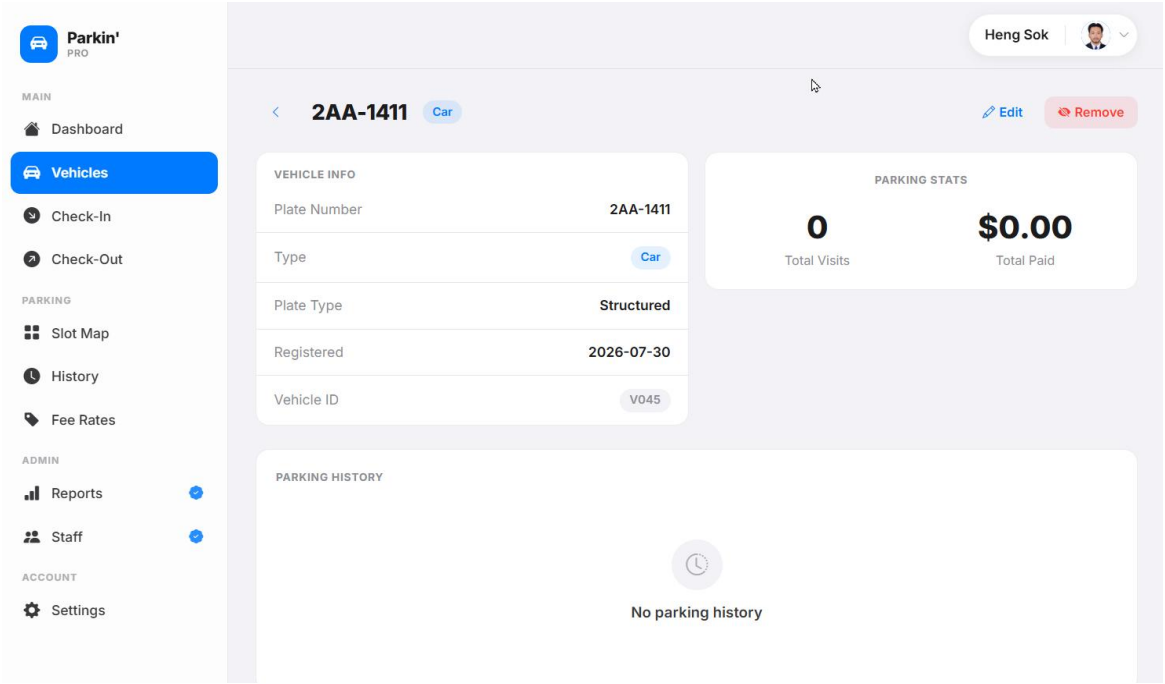


Related Tables	vehicles, tickets
Purpose	Remove a vehicle from the active list. Soft-deletes if it has parking history. Permanently deletes if no history exists.

SQL Statement	`SELECT EXISTS(`
	`  SELECT 1`
	`  FROM  tickets`
	`  WHERE  vehicle_id = 45`
	`) AS has_tickets;`
	
	`UPDATE vehicles`
	`SET  status  = 'deleted',`
	`  updated_at = NOW()`
	`WHERE  vehicle_id = 45;`
	
	`DELETE FROM vehicles`
	`WHERE  vehicle_id = 45;`

Explanation	A confirmation modal appears before deletion. The controller checks for any ticket records. If yes it soft-deletes by setting status = deleted — the vehicle disappears from the list but history is preserved. If no history it is permanently removed.
Expected Result	Vehicle with history disappears from list and history stays intact. Vehicle without history is permanently deleted. Success toast in both cases.

3.6 Vehicle Detail

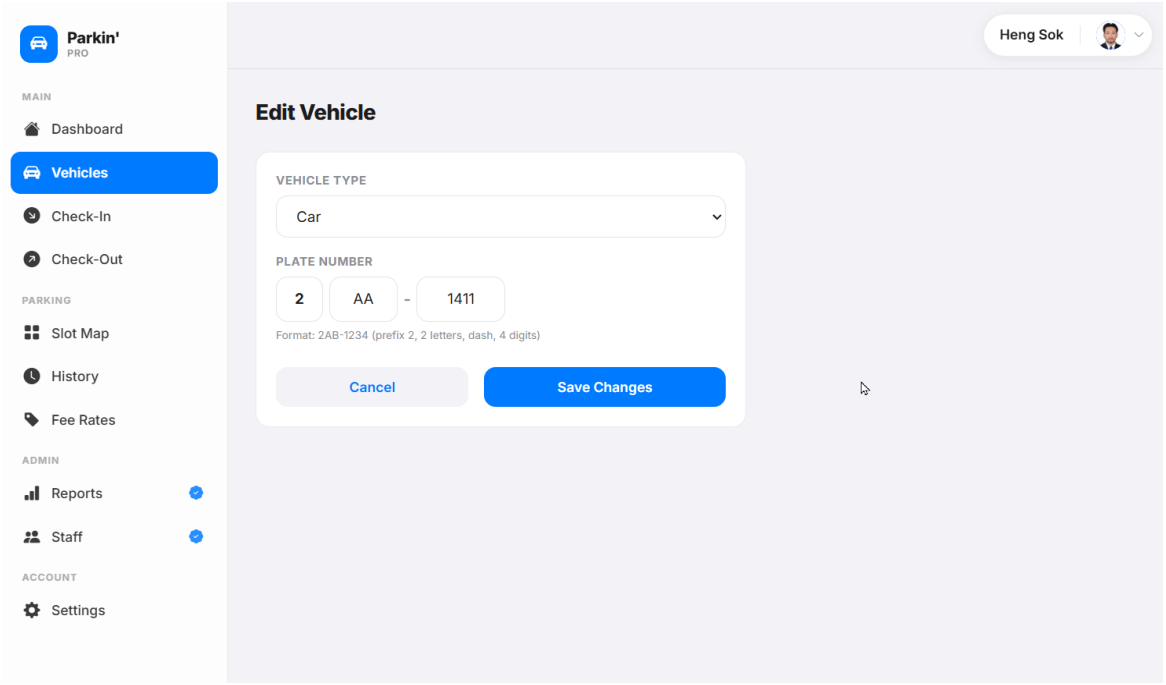


Related Tables	vehicles, tickets, payments
Purpose	View vehicle info, parking stats, and full session history. If currently parked a red panel appears with Check Out and View Ticket buttons.
SQL Statement	

```
SELECT *
FROM   vehicles
WHERE  vehicle_id = 45;

SELECT *
FROM   tickets
WHERE  vehicle_id = 45
  AND  status     = 'active'
LIMIT  1;

SELECT COUNT(*)          AS total_visits
FROM   tickets
WHERE  vehicle_id = 45;

SELECT SUM(p.total_fee) AS total_paid
FROM   payments p
JOIN   tickets  t ON p.ticket_id = t.ticket_id
WHERE  t.vehicle_id = 45;

SELECT t.*, ps.slot_number,
       p.total_fee, p.payment_method, s.full_name
FROM   tickets t
LEFT   JOIN parking_slots ps ON t.slot_id    = ps.slot_id
LEFT   JOIN payments      p  ON t.ticket_id  = p.ticket_id
LEFT   JOIN staff         s  ON t.staff_id   = s.staff_id
WHERE  t.vehicle_id = 45
ORDER  BY t.entry_time DESC;
```

Explanation	Five queries load the vehicle info, check for an active ticket, compute total visits and total paid, and fetch the full session history. The currently parked panel shows estimated fee calculated in PHP from entry_time to now.
Expected Result	Page shows vehicle info, parking stats, currently parked panel if applicable, and full session history with ticket ID, status, slot, entry/exit times, fee, and staff name.

3.7 Edit Vehicle

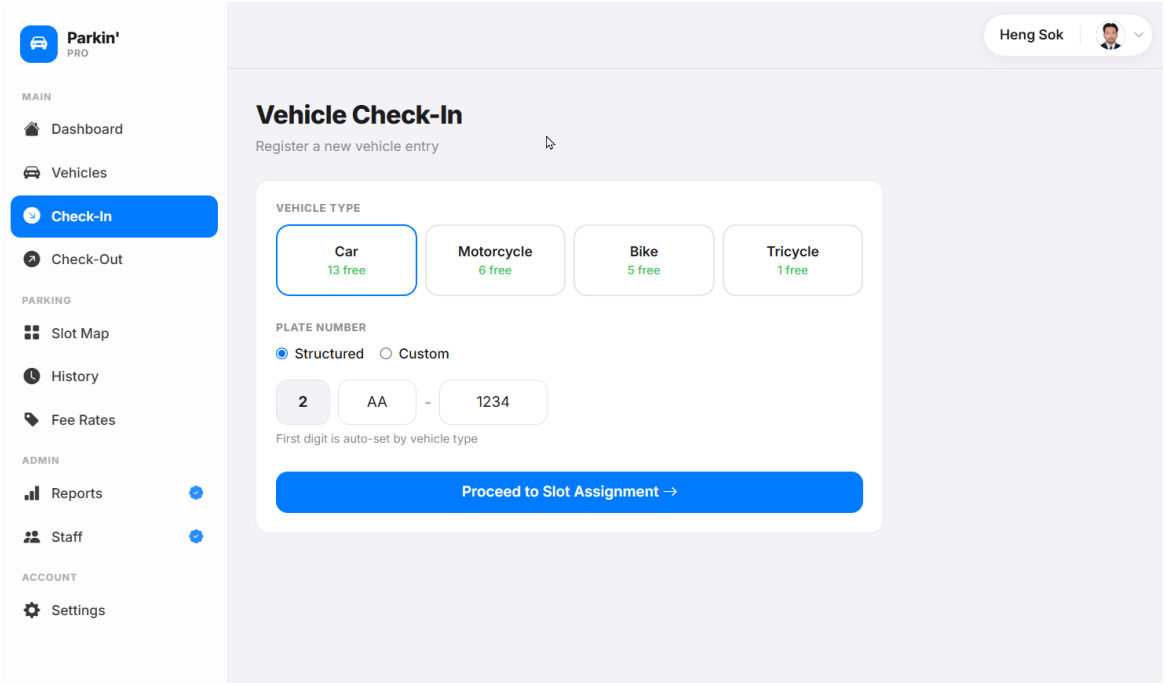


Related Tables	vehicles
Purpose	Edit a vehicle's plate number or type. Structured plate format is enforced on both frontend and server-side.

SQL Statement	`UPDATE vehicles`
	`SET plate_number = '2AA-1411',`
	`vehicle_type = 'car',`
	`updated_at = NOW()`
	`WHERE vehicle_id = 45;`

Explanation	Server-side validation checks that the prefix matches the vehicle type and the full plate matches /^[12][A-Z]{2}-\d{4}$/. Custom plates skip format validation.
Expected Result	Vehicle updated. Redirect to vehicle detail page with Vehicle updated. success message.

3.8 Check-In — Form



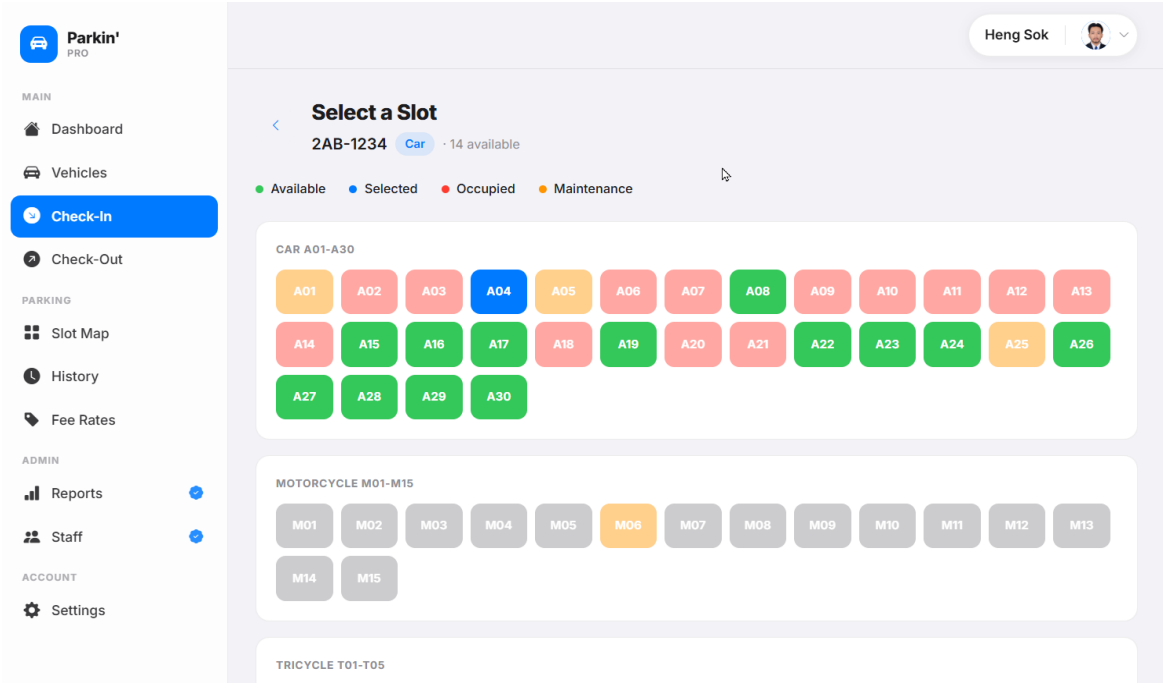
Related Tables	tickets, parking_zones, parking_slots
Purpose	Staff select vehicle type and enter a plate number. Each type card shows live free slot count. Duplicate check prevents the same plate checking in twice.

SQL Statement	`SELECT *`
	`FROM parking_zones;`
	
	`SELECT *`
	`FROM parking_slots`
	`WHERE zone_id IN (1, 2, 3, 4);`
	
	`SELECT slot_id`
	`FROM tickets`
	`WHERE status = 'active';`
	
	`SELECT EXISTS(`
	`SELECT 1`
	`FROM tickets t`
	`JOIN vehicles v ON t.vehicle_id = v.vehicle_id`
	`WHERE t.status = 'active'`
	`AND v.plate_number = '2AB-1234'`
	`) AS already_parked;`

Explanation	Free counts are computed in PHP by excluding occupied slot IDs from active tickets AND maintenance slots. If the plate is already parked the form returns an error before proceeding to slot selection.

Expected Result	Each vehicle type card shows correct free count. Duplicate plate shows error. Valid submission proceeds to slot selection.

3.9 Check-In — Slot Selection



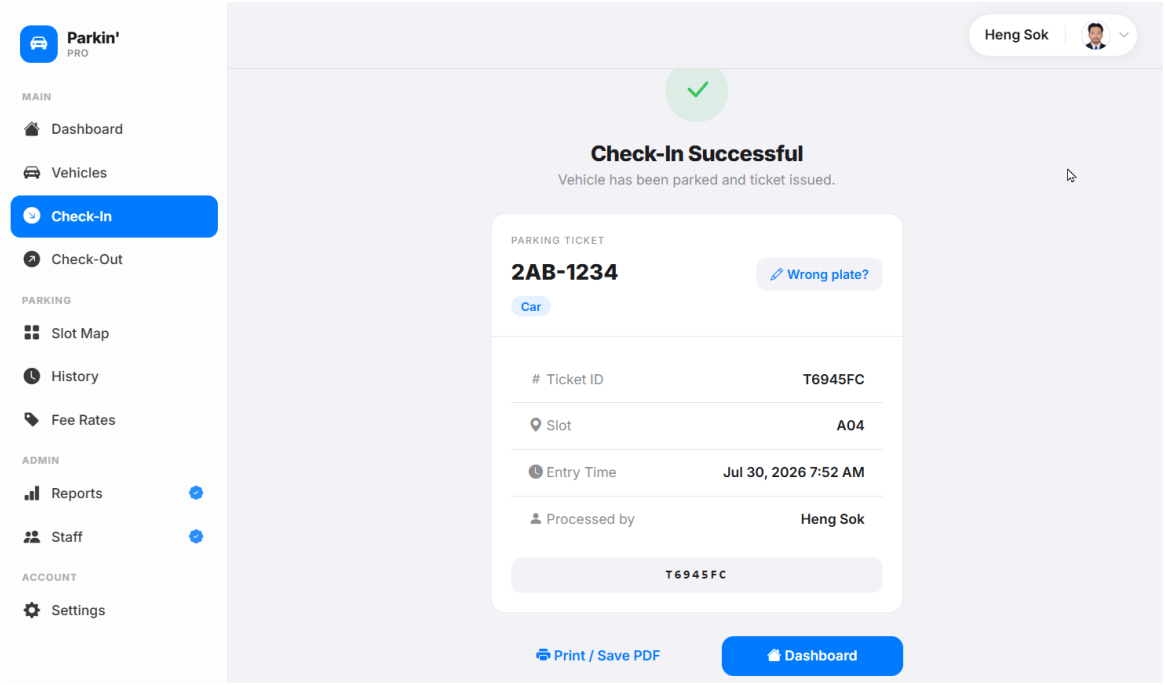
Related Tables	parking_zones, parking_slots, tickets
Purpose	Show the slot grid for all zones. Only the selected vehicle type zone is clickable. Real_status is computed per slot from the active tickets collection.

SQL Statement	`SELECT *`
	`FROM parking_zones;`
	
	`SELECT *`
	`FROM parking_slots`
	`WHERE zone_id IN (1, 2, 3, 4)`
	`ORDER BY slot_number;`
	
	`SELECT *`
	`FROM tickets`
	`WHERE status = 'active';`

Explanation	Active tickets are loaded once and keyed by slot_id in PHP creating a dictionary for O(1) lookup. Per slot: if slot.status is maintenance it shows orange; if zone type does not match the selection it shows grey; otherwise the ticket dictionary determines green or red.

Expected Result	Car zone A01-A30 shows available green, occupied red with plate, and maintenance orange. All other zones are grey and unclickable. Staff click a green slot to confirm.

3.10 Check-In — Ticket Issued



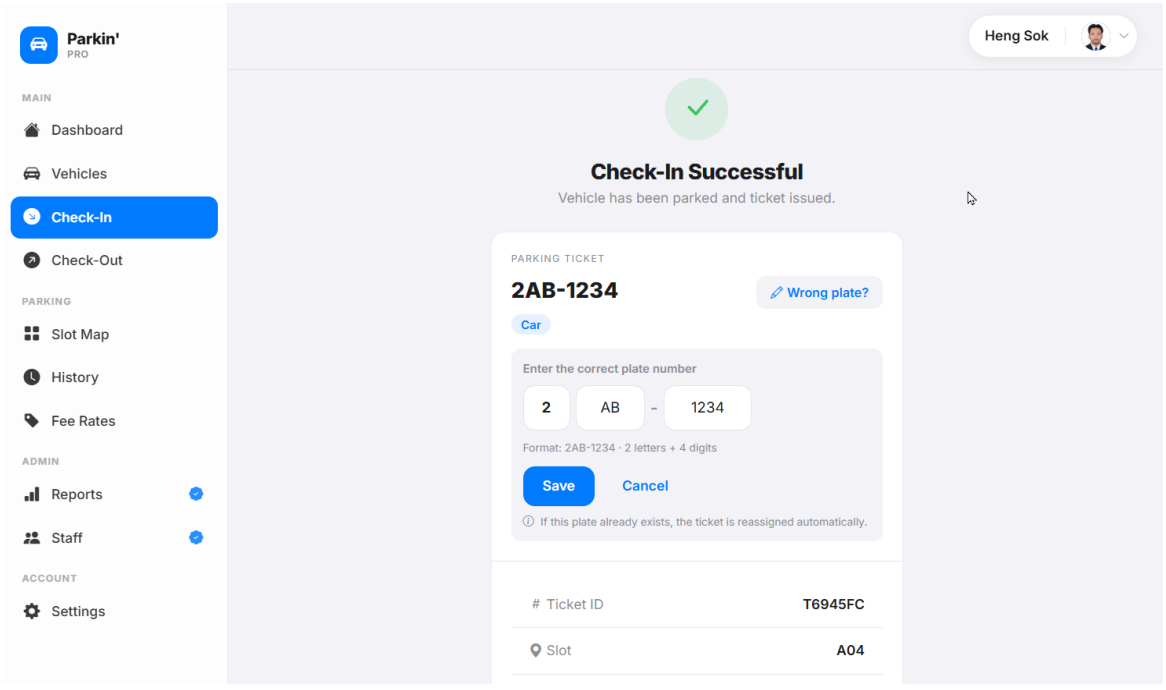
Related Tables	vehicles, parking_slots, tickets, fee_rates
Purpose	Complete check-in inside a database transaction. Create or find the vehicle, mark slot occupied, get fee rate, create ticket with barcode.

SQL Statement	`BEGIN;`
	
	`SELECT *`
	`FROM vehicles`
	`WHERE plate_number = '2AB-1234'`
	`AND vehicle_type = 'car'`
	`AND status = 'active'`
	`LIMIT 1;`
	
	`INSERT INTO vehicles`
	`(plate_number, vehicle_type, plate_type, status,`
	`registered_at, updated_at)`
	`VALUES ('2AB-1234', 'car', 'structured', 'active', NOW(),`
	`NOW());`
	
	`UPDATE parking_slots`
	`SET status = 'occupied',`
	`updated_at = NOW()`
	`WHERE slot_id = 4;`

	<pre>SELECT *
	FROM fee_rates
	WHERE vehicle_type = 'car'
	LIMIT 1;
	
	INSERT INTO tickets
	(ticket_id, vehicle_id, slot_id, staff_id,
	rate_id, entry_time, barcode, status, created_at)
	VALUES ('T6945FC', 1, 4, 1, 1, NOW(), 'T6945FC', 'active',
	NOW());
	
	COMMIT;</pre>

Explanation	All operations run inside DB::transaction(). ticket_id is T plus 6 uppercase chars from PHP's uniqid(). The barcode value equals ticket_id and is rendered as CODE128 using JsBarcode. Any failure triggers ROLLBACK.
Expected Result	Check-In Successful page shows plate, type badge, ticket ID, slot, entry time, processed by, and a scannable barcode. Buttons: Print/Save PDF, Dashboard, Check in another vehicle.

3.11 Correct Wrong Plate



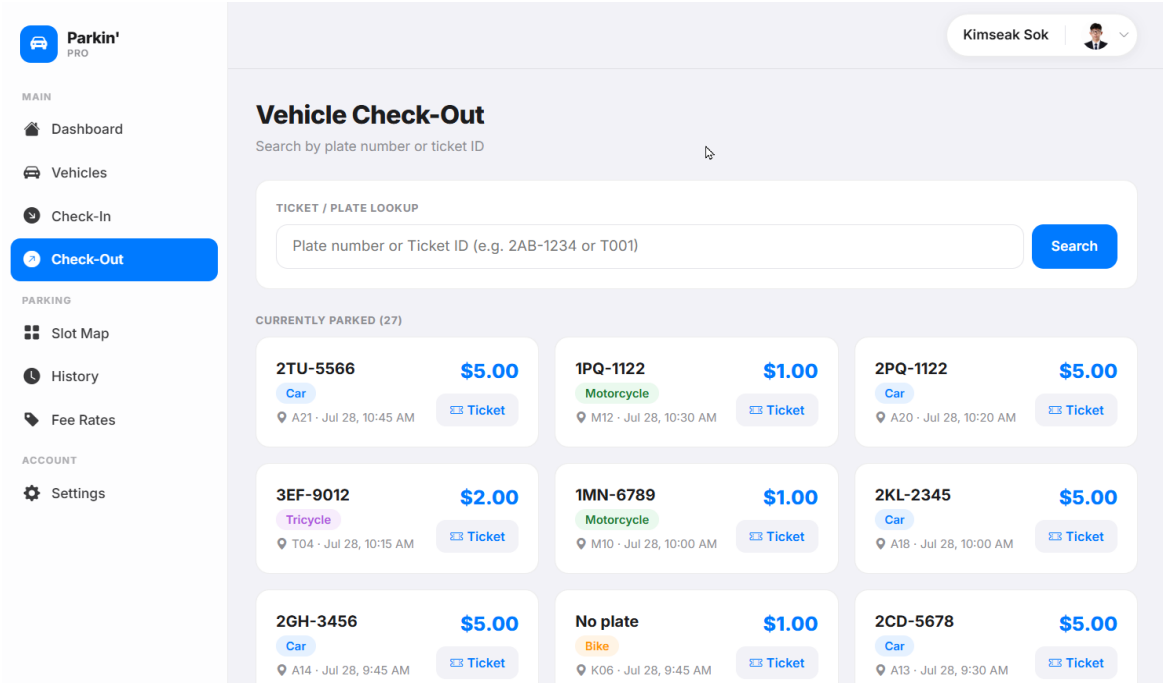
Related Tables	tickets, vehicles

Purpose	Staff correct a wrong plate number from the ticket page after check-in without cancelling. If the correct plate already exists, the ticket is reassigned to that vehicle record.

SQL Statement	`SELECT *`
	`FROM vehicles`
	`WHERE plate_number = '2AB-1234'`
	`AND vehicle_id != 5`
	`LIMIT 1;`
	
	`UPDATE tickets`
	`SET vehicle_id = 7`
	`WHERE ticket_id = 'T6945FC';`
	
	`SELECT EXISTS(`
	`SELECT 1`
	`FROM tickets`
	`WHERE vehicle_id = 5`
	`AND ticket_id != 'T6945FC'`
	`) AS has_other;`
	
	`DELETE FROM vehicles`
	`WHERE vehicle_id = 5;`
	
	`UPDATE vehicles`
	`SET plate_number = '2AB-1234'`
	`WHERE vehicle_id = 5;`

Explanation	If the correct plate belongs to an existing vehicle, the ticket FK is reassigned and the wrong vehicle is deleted if it has no other history. If the plate is new, only the plate text is updated. Slot, entry time, and ticket ID remain unchanged.
Expected Result	Ticket page reloads showing corrected plate. Success toast Plate corrected to 2AB-1234. appears at the top.

3.12 Vehicle Search — Currently Parked

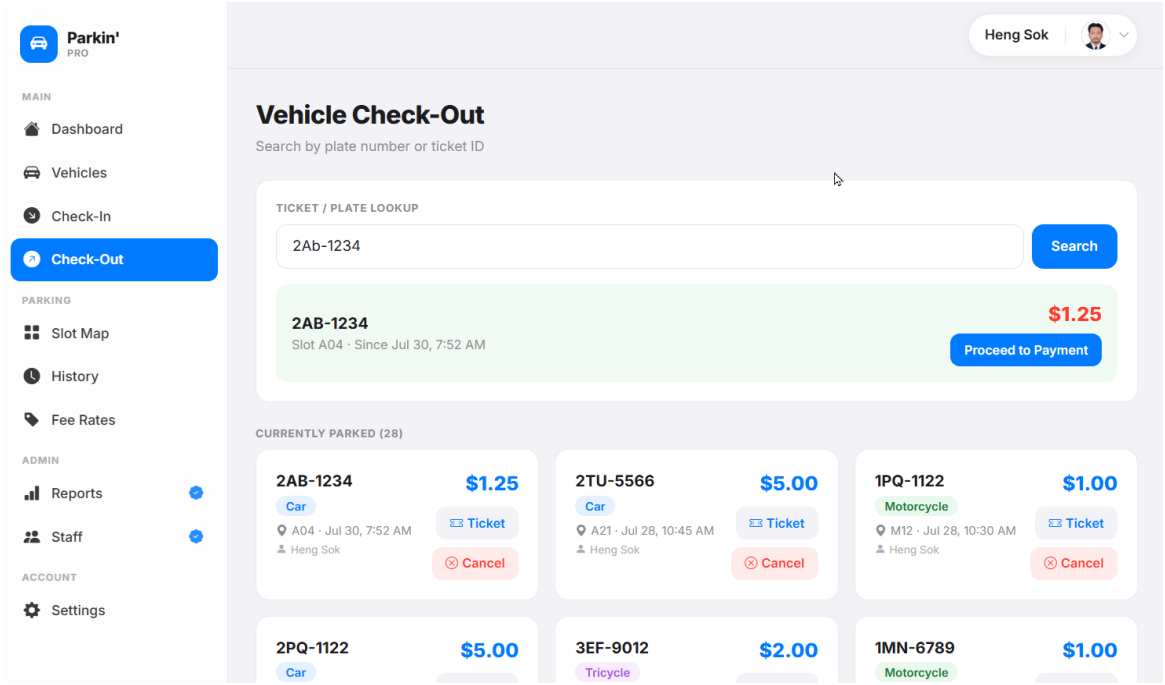


Related Tables	vehicles, tickets, fee_rates, parking_slots
Purpose	When staff search by plate on the Vehicles page, a red Currently Parked panel appears if the vehicle is active, showing slot, duration, estimated fee, and action buttons.

SQL Statement	`SELECT * FROM vehicles WHERE status = 'active' AND plate_number LIKE '%2AA-1411%';`
	
	`SELECT * FROM tickets WHERE status = 'active' AND EXISTS ( SELECT 1 FROM vehicles WHERE vehicles.vehicle_id = tickets.vehicle_id AND vehicles.plate_number LIKE '%2AA-1411%' ) LIMIT 1;`

Explanation	The vehicle list search runs a parallel active ticket lookup. The EXISTS subquery checks if any active ticket belongs to a matching vehicle. Estimated fee is computed in PHP from entry_time to now using the fee rate. Staff can proceed directly to payment without needing the ticket ID.
Expected Result	Vehicle list shows matching result. Red Currently Parked panel shows slot, entry time, duration, estimated fee, and three buttons: View Ticket, Edit Plate, Check Out.

3.13 Check-Out — Search

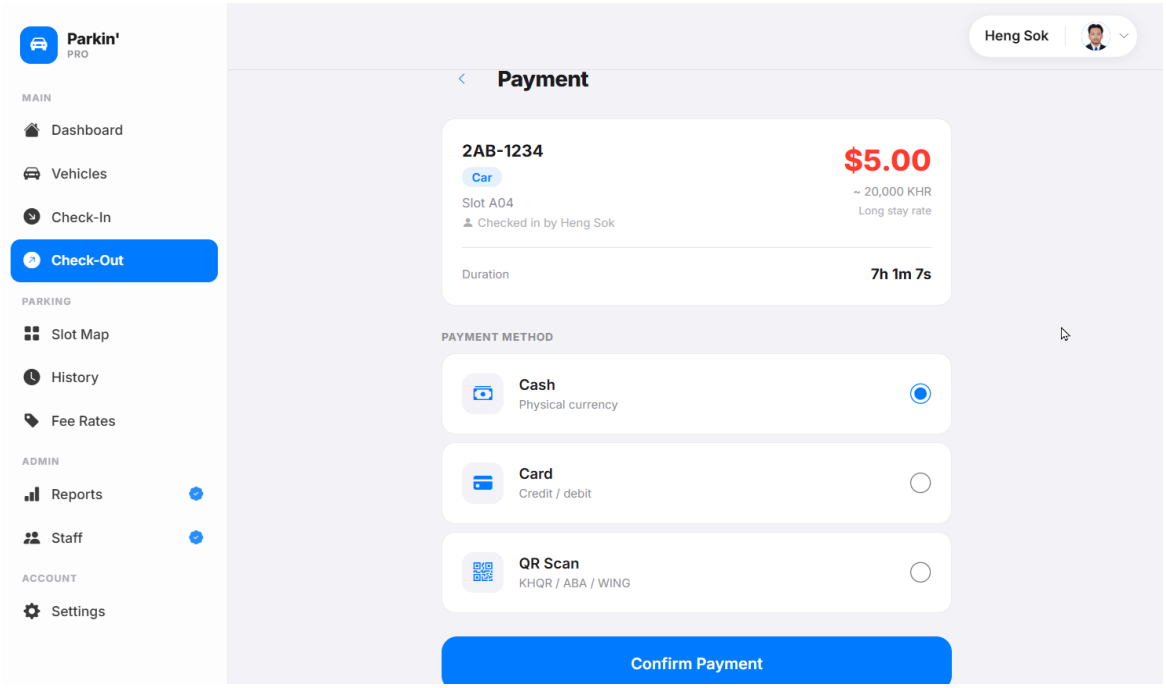


Related Tables	tickets, vehicles, parking_slots, fee_rates
Purpose	Search for an active parking session by ticket ID (exact) or plate number (whereHas). Estimated fee shown in the result panel.

SQL Statement	`SELECT * FROM tickets WHERE status = 'active' AND ticket_id = 'T6945FC' LIMIT 1;`
	
	`SELECT * FROM tickets WHERE status = 'active' AND EXISTS ( SELECT 1 FROM vehicles WHERE vehicles.vehicle_id = tickets.vehicle_id AND vehicles.plate_number = '2AB-1234' ) LIMIT 1;`

Explanation	Ticket ID is checked first with an exact match. If not found, plate number is checked using an EXISTS subquery. This handles lost-ticket scenarios where only the plate is known. Estimated fee is calculated in PHP from the entry_time.
Expected Result	Result panel shows vehicle plate, slot, entry time since, and estimated fee with Proceed to Payment button.

3.14 Check-Out — Payment

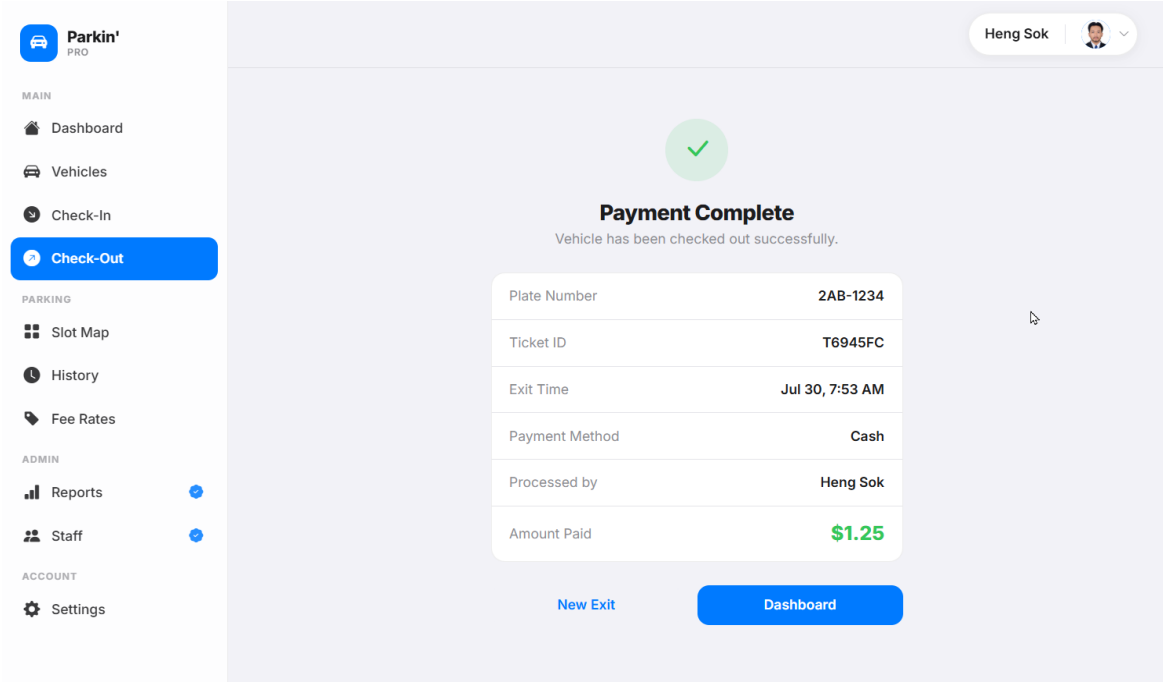


Related Tables	tickets, fee_rates, payments, parking_slots
Purpose	Show live fee with timer and payment method selection. Fee is recalculated server-side at confirm to prevent browser manipulation.
SQL Statement	

```
BEGIN;

SELECT *
FROM   tickets
WHERE  ticket_id = 'T6945FC'
  AND  status    = 'active'
FOR    UPDATE;

UPDATE tickets
SET    status    = 'completed',
       exit_time = NOW()
WHERE  ticket_id = 'T6945FC';

UPDATE parking_slots
SET    status     = 'available',
       updated_at = NOW()
WHERE  slot_id    = 4;

INSERT INTO payments
       (ticket_id, staff_id, duration,
        total_fee, payment_method, paid_at)
VALUES ('T6945FC', 1, 0.02, 1.25, 'cash', NOW());
```

	`COMMIT;`
Explanation	FOR UPDATE locks the ticket row to prevent double-processing. Fee is recalculated at commit time from the real exit timestamp — the browser timer is display only. Three atomic operations: ticket completed, slot freed, payment inserted. Any failure triggers ROLLBACK.
Expected Result	Payment Complete page shows plate, ticket ID, exit time, payment method, staff name, and amount paid in green. Buttons: New Exit, Dashboard.

3.15 Check-Out — Complete

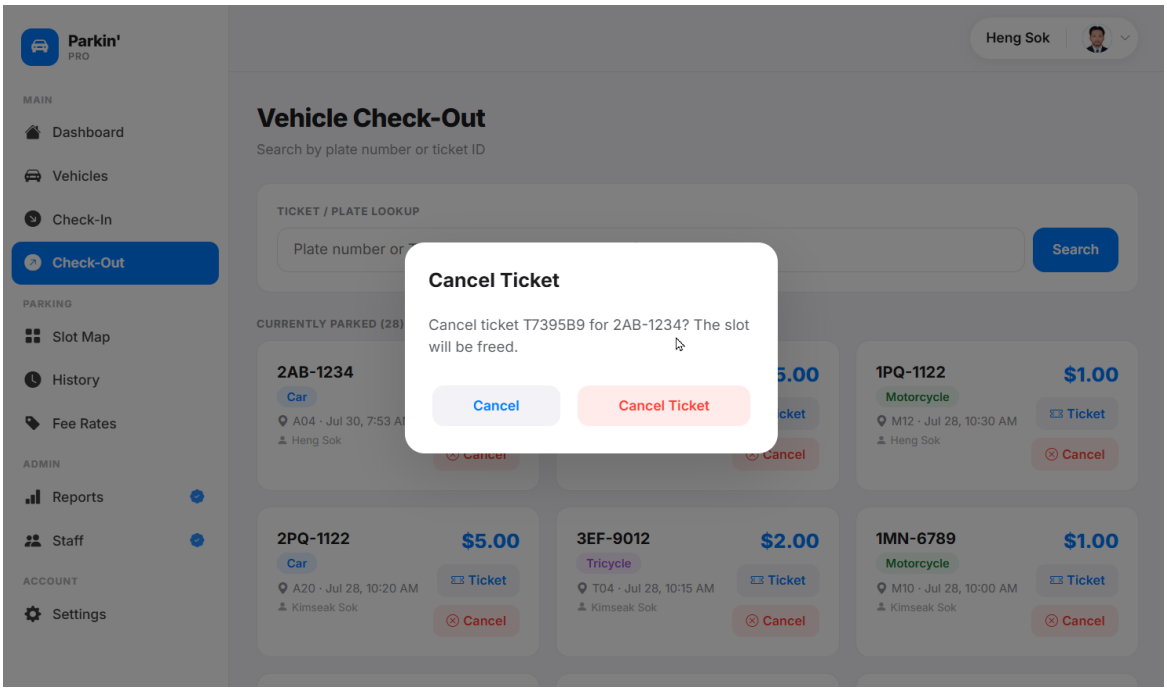


Related Tables	tickets, payments, vehicles, staff
Purpose	Show the payment receipt after a successful checkout.

SQL Statement	`SELECT t.*, v.plate_number, v.vehicle_type,`
	`p.total_fee, p.payment_method, p.paid_at,`
	`s.full_name AS processed_by`
	`FROM tickets t`
	`JOIN vehicles v ON t.vehicle_id = v.vehicle_id`
	`LEFT JOIN payments p ON t.ticket_id = p.ticket_id`
	`LEFT JOIN staff s ON t.staff_id = s.staff_id`
	`WHERE t.ticket_id = 'T6945FC';`

Explanation	Loads the completed ticket with all related data for the receipt display using LEFT JOINs to handle the unlikely case of a missing payment or staff record.
Expected Result	Receipt shows: Plate 2AB-1234, Ticket T6945FC, Exit Time Jul 30 7:53 AM, Payment Method Cash, Processed by Heng Sok, Amount Paid $1.25.

3.16 Cancel Ticket (Admin)

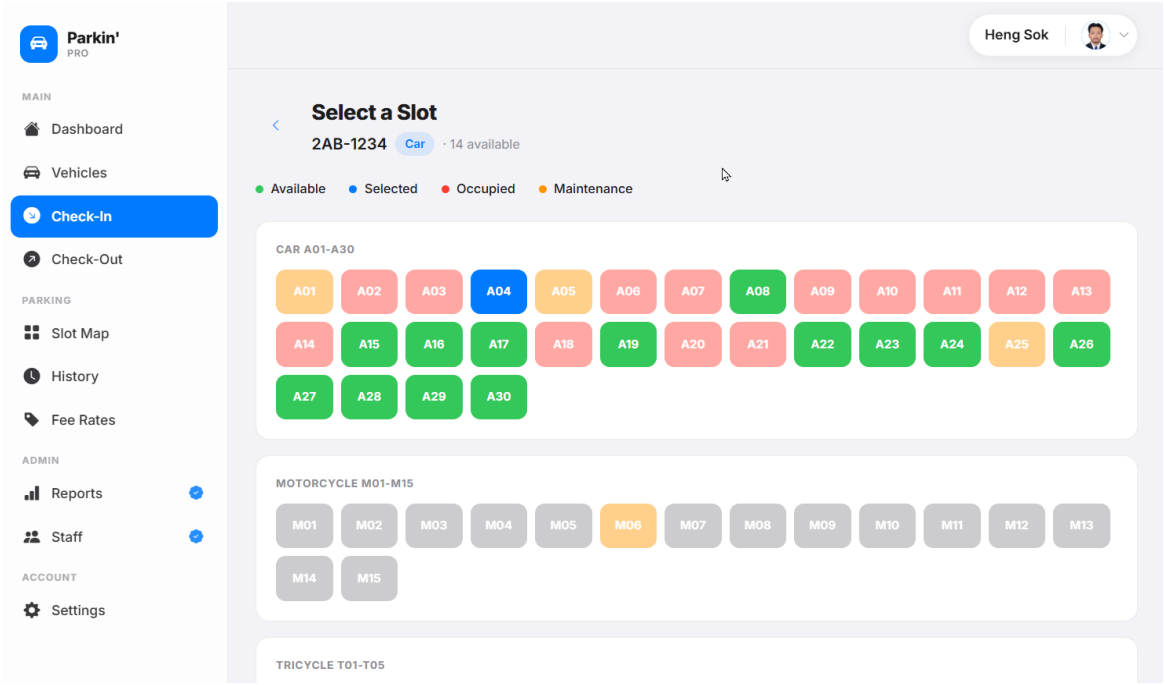


Related Tables	tickets, parking_slots
Purpose	Admin cancels an active ticket from the checkout page. No payment is recorded. The slot is freed immediately.

SQL Statement	`BEGIN;`
	
	`UPDATE tickets`
	`SET status = 'cancelled',`
	`exit_time = NOW()`
	`WHERE ticket_id = 'T7395B9'`
	`AND status = 'active';`
	
	`UPDATE parking_slots`
	`SET status = 'available',`
	`updated_at = NOW()`
	`WHERE slot_id = 4;`
	
	`COMMIT;`

Explanation	A confirmation modal appears before cancellation. The cancel button is only visible to admin on the checkout page. Two atomic updates run in a transaction. No payment record is created.
Expected Result	Redirect to Slot Map with success toast Ticket T7395B9 cancelled and slot freed. The slot immediately shows as available.

3.17 Slot Map — Aerial View

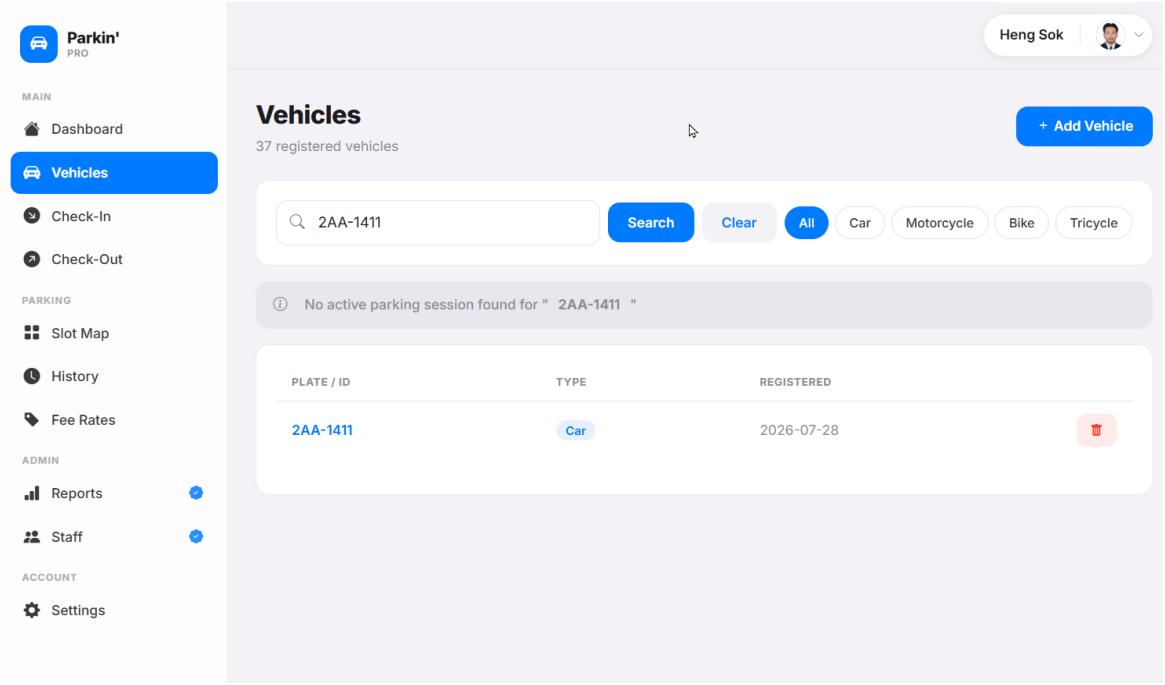


Related Tables	parking_zones, parking_slots, tickets, vehicles
Purpose	Show all 58 slots as a colour-coded grid grouped by zone. Summary stat cards show total, available, occupied, and maintenance counts.

SQL Statement	`SELECT *`
	`FROM parking_zones;`
	
	`SELECT *`
	`FROM parking_slots`
	`WHERE zone_id IN (1, 2, 3, 4)`
	`ORDER BY slot_number;`
	
	`SELECT t.*, v.plate_number, v.vehicle_type`
	`FROM tickets t`
	`JOIN vehicles v ON t.vehicle_id = v.vehicle_id`
	`WHERE t.status = 'active';`

Explanation	Four queries total. Active tickets are keyed by slot_id in PHP for O(1) lookup. Real_status per slot: maintenance first from slot.status, then ticket dictionary check. Summary counts computed from PHP collections — no extra SQL.
Expected Result	Grid shows all 58 slots across 4 zones. Available green, occupied red with plate number inside, maintenance orange. Stat cards show correct counts at the top.

3.18 Slot Map — List View

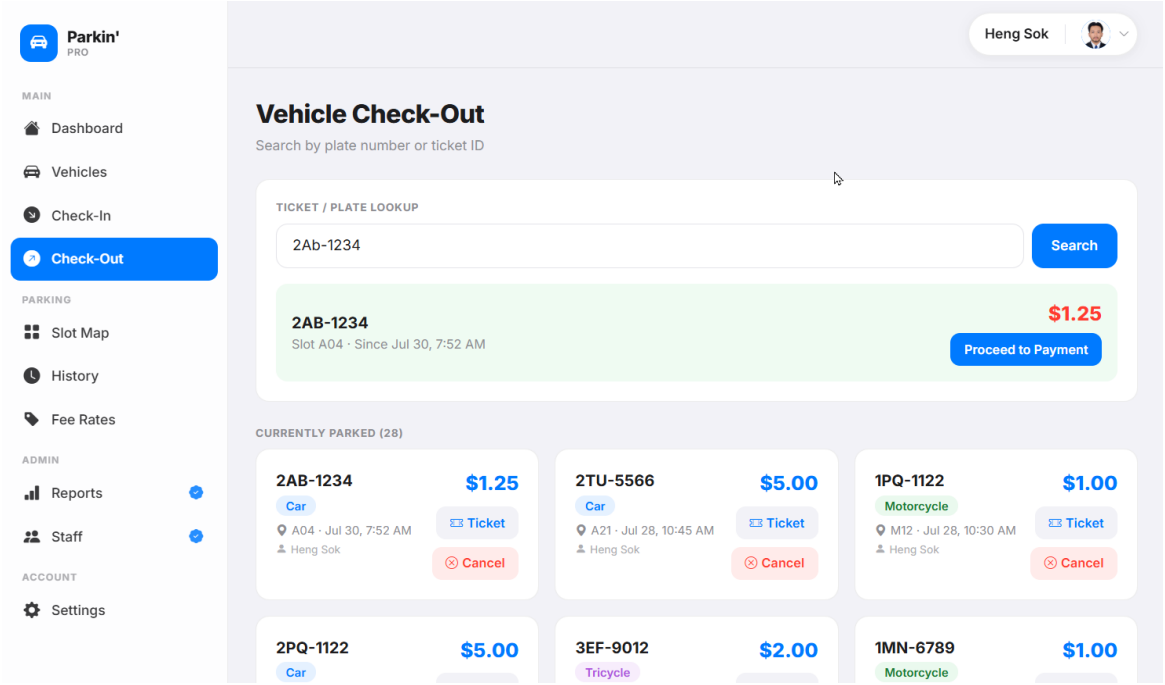


Related Tables	parking_zones, parking_slots, tickets, vehicles
Purpose	Show all slots in a searchable list. Occupied rows show plate, type, and inline Check Out button. Search filters by slot number or plate in real time with no extra SQL.

SQL Statement	`SELECT *`
	`FROM parking_zones;`
	
	`SELECT *`
	`FROM parking_slots`
	`WHERE zone_id IN (1, 2, 3, 4)`
	`ORDER BY slot_number;`
	
	`SELECT t.*, v.plate_number, v.vehicle_type`
	`FROM tickets t`
	`JOIN vehicles v ON t.vehicle_id = v.vehicle_id`
	`WHERE t.status = 'active';`

Explanation	The list view reuses the same controller data as the aerial view — same 3 queries. Client-side JavaScript filters rows by matching data-slot and data-plate attributes on each table row element. No database calls on search.
Expected Result	Searching 2AB shows only matching rows in real time. Occupied rows show Edit Plate pencil and Check Out red button inline. Available and maintenance rows show a chevron to the slot detail page.

3.19 Slot Detail and Maintenance Toggle



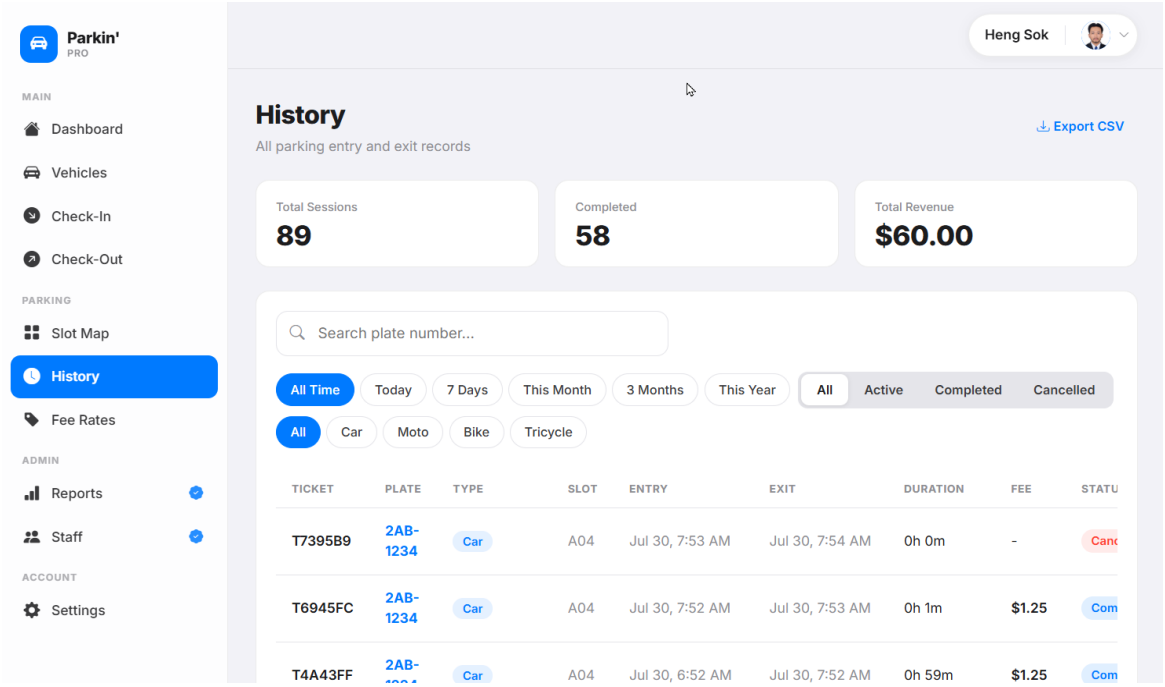
Related Tables	parking_slots, parking_zones, tickets
Purpose	Show detailed info for a single slot including real_status, total uses, and currently parked vehicle. Admin can toggle maintenance mode.

SQL Statement	`SELECT *`
	`FROM parking_slots`
	`WHERE slot_id = 5`
	`LIMIT 1;`
	
	`SELECT *`
	`FROM parking_zones`
	`WHERE zone_id = 1`
	`LIMIT 1;`
	
	`SELECT *`
	`FROM tickets`

	<pre>WHERE slot_id = 5
	  AND status = 'active'
	LIMIT 1;
	
	SELECT COUNT(*)
	FROM tickets
	WHERE slot_id = 5;
	
	SELECT EXISTS(
	  SELECT 1
	  FROM tickets
	  WHERE slot_id = 5
	  AND status = 'active'
	) AS has_ticket;
	
	UPDATE parking_slots
	SET status = 'maintenance',
	  updated_at = NOW()
	WHERE slot_id = 5;</pre>

Explanation	Route model binding auto-fetches the slot. Real_status is computed from the ticket table — if maintenance, no ticket query runs. Cannot toggle a slot to maintenance if occupied. The page also hosts the Wrong plate? correction form.
Expected Result	Slot detail shows slot number, zone, type badge, real_status pill, total uses. Occupied slots show the vehicle panel with entry time, duration, estimated fee, staff. Admin sees maintenance toggle and Check Out shortcut.

3.20 History

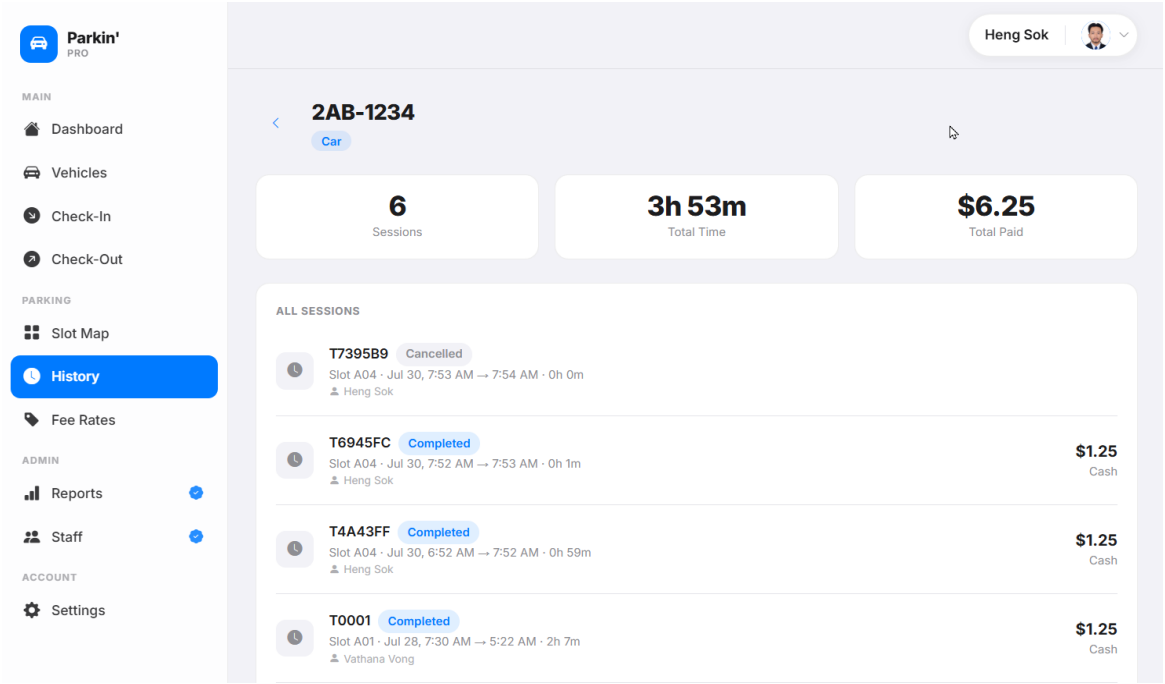


Related Tables	tickets, vehicles, parking_slots, payments, staff
Purpose	Full log of all parking sessions with period, status, and vehicle type filters. Searchable by plate. CSV export available.

SQL Statement	`SELECT COUNT(*) AS total FROM tickets;`
	`SELECT COUNT(*) AS completed FROM tickets WHERE status = 'completed';`
	`SELECT SUM(total_fee) AS revenue FROM payments;`
	
	`SELECT t.ticket_id, v.plate_number, v.vehicle_type,`
	`ps.slot_number, t.entry_time, t.exit_time,`
	`t.status, p.total_fee, p.payment_method,`
	`s.full_name`
	`FROM tickets t`
	`JOIN vehicles v ON t.vehicle_id = v.vehicle_id`
	`LEFT JOIN parking_slots ps ON t.slot_id = ps.slot_id`
	`LEFT JOIN payments p ON t.ticket_id = p.ticket_id`
	`LEFT JOIN staff s ON t.staff_id = s.staff_id`
	`WHERE t.entry_time >= '2026-07-23 00:00:00'`
	`AND t.status = 'completed'`
	`AND v.vehicle_type = 'car'`
	`ORDER BY t.entry_time DESC`
	`LIMIT 20 OFFSET 0;`

Explanation	Summary stats always count all-time totals regardless of filter. The filtered query builds dynamically based on active period, status, and type filters. Search mode bypasses all filters and uses a LIKE on plate_number only.
Expected Result	Table shows matching sessions with ticket ID, plate, type, slot, entry, exit, duration, fee, status pill. Summary cards: 89 total, 58 completed, $60.00 revenue.

3.21 Vehicle History by Plate

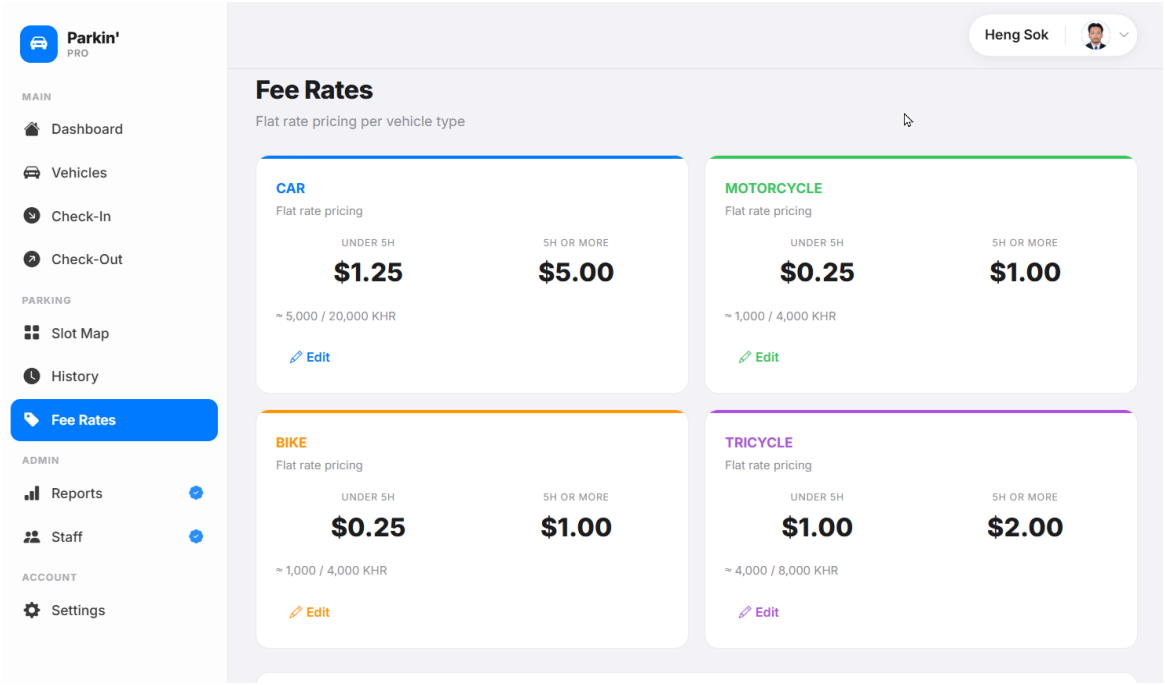


Related Tables	vehicles, tickets, payments, parking_slots, staff
Purpose	Clicking a plate number in history loads all sessions for that vehicle with aggregate stats.

SQL Statement	`SELECT *`
	`FROM vehicles`
	`WHERE plate_number = '2AB-1234'`
	`AND status = 'active'`
	`LIMIT 1;`
	
	`SELECT t.ticket_id, t.entry_time, t.exit_time,`
	`t.status, ps.slot_number,`
	`p.total_fee, p.payment_method, s.full_name`
	`FROM tickets t`
	`LEFT JOIN parking_slots ps ON t.slot_id = ps.slot_id`
	`LEFT JOIN payments p ON t.ticket_id = p.ticket_id`
	`LEFT JOIN staff s ON t.staff_id = s.staff_id`
	`WHERE t.vehicle_id = 1`
	`ORDER BY t.entry_time DESC;`

Explanation	Vehicle is fetched by plate first. Then all sessions are loaded with a LEFT JOIN to payments and staff. PHP aggregates total sessions, total minutes, and total paid from the collection without extra SQL.
Expected Result	Vehicle history shows: 2AB-1234 Car, 6 Sessions, 3h 53m total time, $6.25 total paid, and all sessions listed with status, slot, times, fee, and payment method.

3.22 Fee Rates

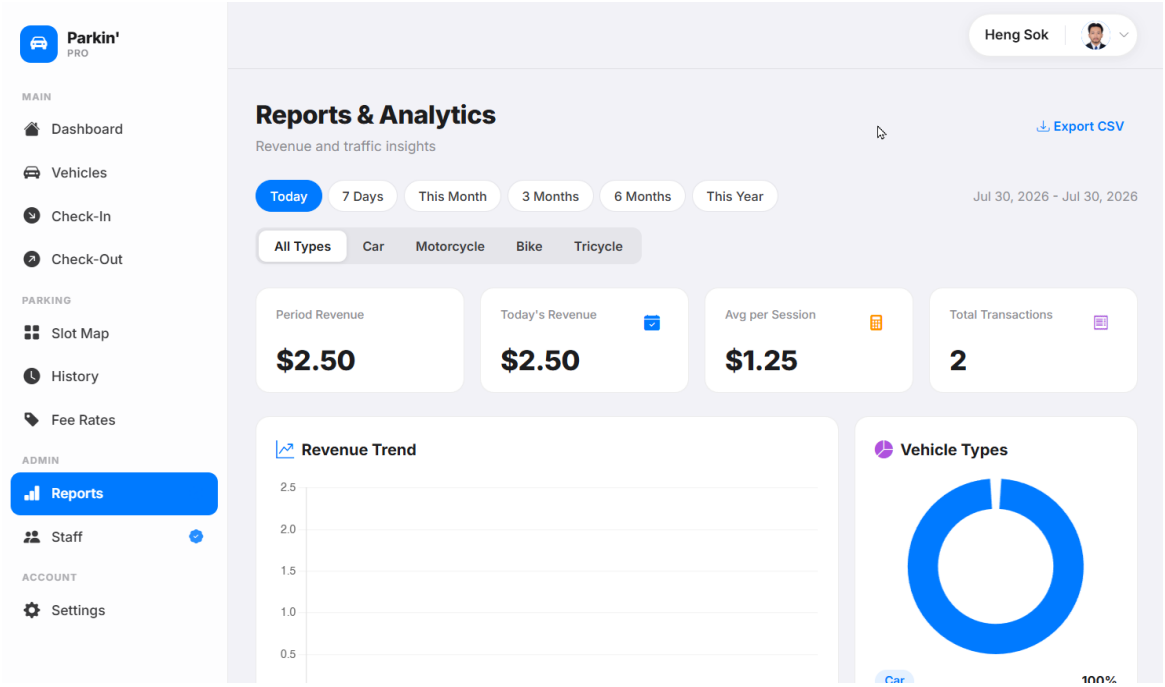


Related Tables	fee_rates
Purpose	View and edit flat-rate fees per vehicle type. Short stay under 5 hours and long stay at 5 hours or more. Displayed in USD and KHR.

SQL Statement	`SELECT *`
	`FROM fee_rates;`
	
	`UPDATE fee_rates`
	`SET short_stay_fee = 1.50,`
	`long_stay_fee = 6.00,`
	`updated_at = NOW()`
	`WHERE rate_id = 1;`

Explanation	Rates are loaded keyed by vehicle_type for Blade rendering. Each card shows USD amounts and KHR equivalent (1 USD = 4,000 KHR). Admin clicks Edit to reveal inline input fields for the two fee values.
Expected Result	Four rate cards: Car $1.25/$5.00, Motorcycle $0.25/$1.00, Bike $0.25/$1.00, Tricycle $1.00/$2.00. Each with KHR equivalent. Admin can edit and save each card independently.

3.23 Reports

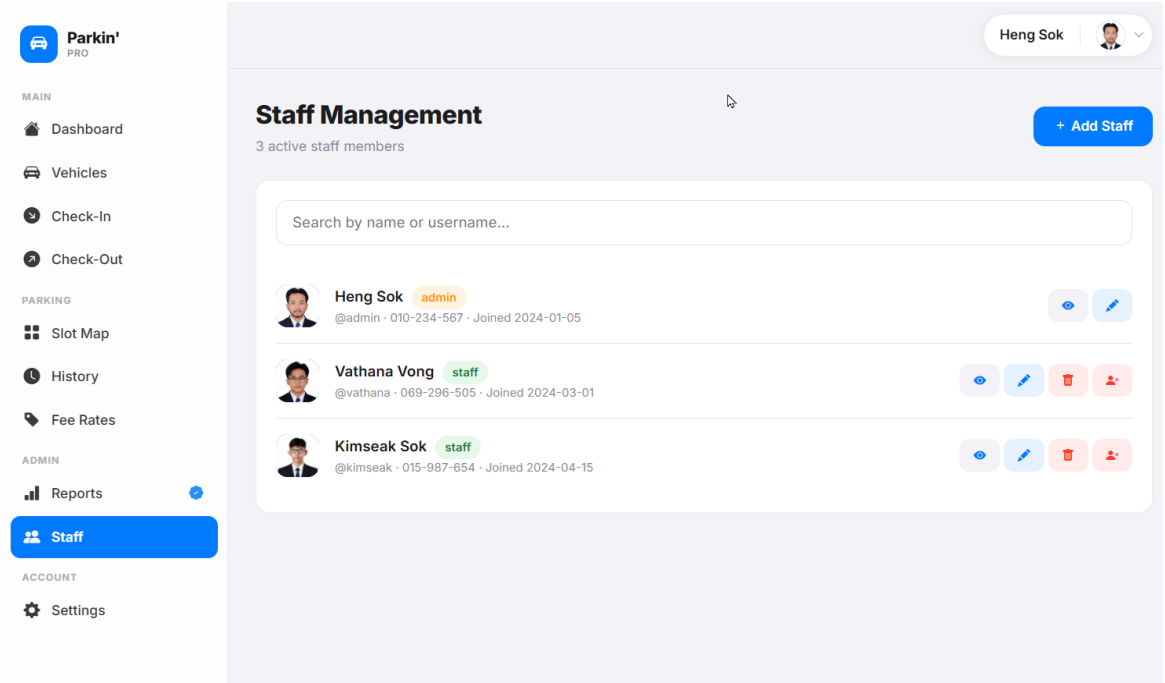


Related Tables	payments, tickets, vehicles
Purpose	Admin revenue analytics. Period and vehicle type filters. Revenue trend chart, type breakdown, and top 5 vehicles by visit count.
SQL Statement	

```
SELECT SUM(p.total_fee)  AS revenue,
       COUNT(p.payment_id) AS transactions
FROM   payments p
JOIN   tickets  t ON p.ticket_id  = t.ticket_id
JOIN   vehicles v ON t.vehicle_id = v.vehicle_id
WHERE  p.paid_at         >= '2026-07-01 00:00:00'
  AND  v.vehicle_type     = 'car';

SELECT DATE(p.paid_at) AS date,
       SUM(p.total_fee) AS total
FROM   payments p
JOIN   tickets t ON p.ticket_id = t.ticket_id
WHERE  p.paid_at >= '2026-07-01'
GROUP  BY date
ORDER  BY date;

SELECT v.plate_number,
       v.vehicle_type,
       (SELECT COUNT(*)
        FROM   tickets t
        WHERE  t.vehicle_id = v.vehicle_id) AS visit_count,
       (SELECT COALESCE(SUM(p.total_fee), 0)
        FROM   payments p
        JOIN   tickets  t2 ON t2.ticket_id = p.ticket_id
        WHERE  t2.vehicle_id = v.vehicle_id) AS total_spent
FROM   vehicles v
WHERE  v.status = 'active'
HAVING visit_count > 0
```

	`ORDER BY visit_count DESC` `LIMIT 5;`
Explanation	Revenue is qualified as payments.total_fee after JOIN to avoid ambiguous column errors. Daily trend uses DATE() and GROUP BY for the line chart. Top 5 uses correlated subqueries to get visit count and total spent per vehicle.
Expected Result	Shows period revenue, today's revenue, avg per session, total transactions. Revenue trend line chart, vehicle type donut chart, and top 5 vehicles table below.

3.24 Staff Management



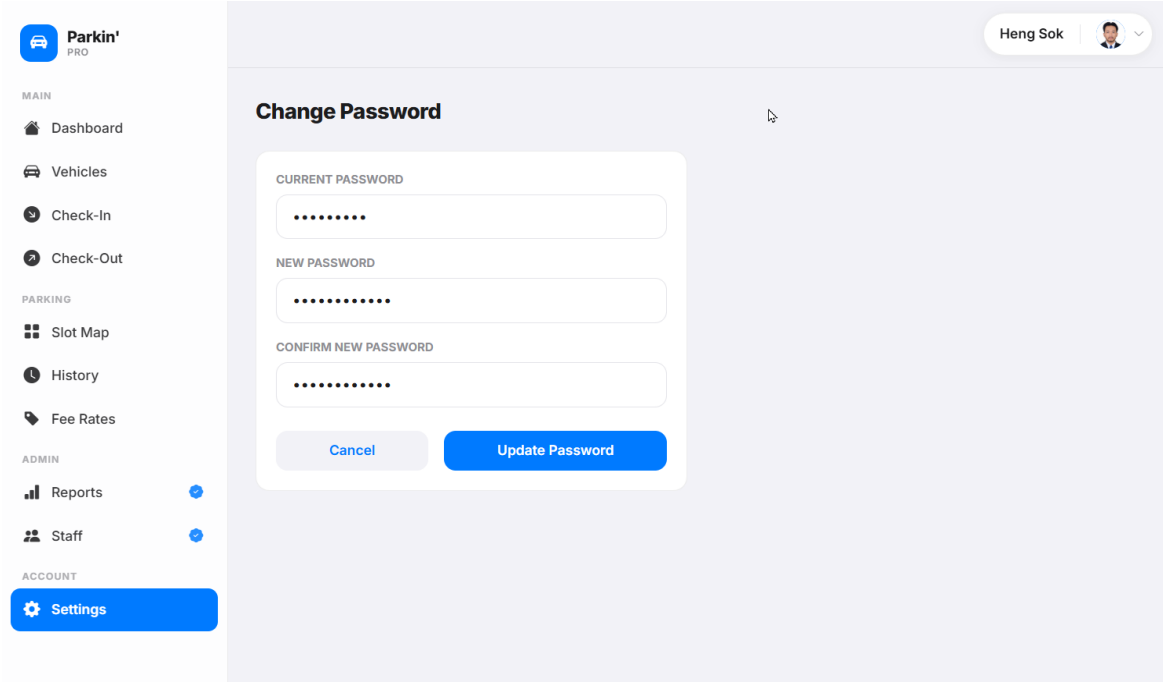
Related Tables	staff
Purpose	Admin views, adds, edits, deactivates, and deletes staff accounts. Cannot delete own account. New staff default password is password.

SQL Statement	`SELECT *`
	`FROM staff`
	`ORDER BY created_at ASC;`
	
	`INSERT INTO staff`
	`(username, password_hash, full_name, gender, role,`
	`phone_number, status, date_of_birth,`
	`created_at, updated_at)`
	`VALUES ('staff_kimseng', '$2y$12$...', 'Kimseng Sok',`
	`'male', 'staff', '069-66-11-88',`
	`'active', '2026-07-30', NOW(), NOW());`

	`UPDATE staff`
	`SET status = CASE`
	`WHEN status = 'active' THEN 'deactivated'`
	`ELSE 'active'`
	`END`
	`WHERE staff_id = 5;`
	
	`DELETE FROM staff`
	`WHERE staff_id = 5;`

Explanation	New staff receive a bcrypt-hashed default password. Deactivating sets status = deactivated blocking login while preserving the account and all activity records. Admin cannot delete themselves — the controller checks auth()->id() before proceeding.
Expected Result	Staff list shows all members with name, username, role badge, join date, and action buttons. Add, edit, deactivate, and delete all work independently with confirmation modals.

3.25 Settings — Change Password



Related Tables	staff
Purpose	Any staff member can change their own password. Requires current password verification before the hash is updated.

SQL Statement	`UPDATE staff`
	`SET password_hash = '$2y$12$newHashHere...',`
	`updated_at = NOW()`
	`WHERE staff_id = 1;`

Explanation	Current password is verified using Laravel's Hash::check() against password_hash before the UPDATE runs. The new password is bcrypt-hashed before storage. This form is available to both staff and admin.
Expected Result	On correct current password: success toast Password updated. On wrong current password: error Current password is incorrect. Page stays on change password screen.

Conclusion

Parkin' covers the full parking lifecycle from vehicle entry to payment collection and financial reporting. The database design follows normalization principles with 7 tables, explicit foreign key constraints, and indexes on frequently queried columns.

Key Engineering Decisions

Real_status pattern: Slot occupancy is always derived from active tickets, never from slot.status which can go stale. This fix applies in three places: check-in free count, check-in slot grid, and the slot map.

Atomic transactions: All write operations run inside DB::transaction() with lockForUpdate() to prevent partial state and race conditions.

Zone-based slot design: vehicle_type lives on parking_zones based on professor feedback. A slot inherits its accepted vehicle type from its zone via JOIN.

Efficient SQL: GROUP BY HOUR() for the dashboard chart reduces 24 queries to 1. keyBy(slot_id) gives O(1) slot lookup. Correlated subqueries handle top 5 vehicles. Qualified column names prevent ambiguous column errors in multi-table JOINs.

Audit trail: Vehicles and tickets with history are soft-deleted. No associated data is ever permanently lost unless there is no linked history.